Goa Business School
Department of Computer Science & Technology

MINI PROJECT REPORT

Object Oriented Technology (OOT)

LIVE TRANSIT TRACKING

Bus Tracker: Real-Time Public Bus Tracking for Goa

(KTCL and Private Buses using Driver Smartphones)

Group B3G2

Student Name	Roll Number
Saish Gawade	2641
Prathamesh Rane	2635

Supervisor / Guide: Assistant Professor Hanumant Harichandra Redkar

Academic Year: 2026 – 2027

Date of Submission: 26 October 2026

Declaration

We hereby declare that the mini project report entitled **"Live Transit Tracking – Bus Tracker"**, submitted for the course Object Oriented Technology at Goa Business School , is a record of original work carried out by us under the guidance of Assistant Professor Hanumant Harichandra Redkar. The work has not been submitted to any other institution or for any other degree, diploma or course.

Wherever the work of others has been used, it has been duly acknowledged in the text and in the list of references.

Saish Gawade (2641) Prathamesh Rane (2635)

Date: 26 October 2026

Place: Goa Business School

Acknowledgements

We express our sincere gratitude to our project guide, Assistant Professor Hanumant Harichandra Redkar, for continuous guidance, encouragement and constructive feedback throughout this project. We thank the Program Director and the faculty of Department of Computer Science & Technology for providing the facilities and the academic environment required to complete this work.

We also thank the open-source communities behind Spring Boot, Android, Jetpack Compose, osmdroid, OkHttp and OpenStreetMap, whose free tools and documentation made this project possible. Finally, we thank our friends, families and the commuters who shared their experiences of public transport in Goa, which shaped the problem we set out to solve.

Saish Gawade (2641) Prathamesh Rane (2635)

Abstract / Executive Summary

Public bus transport in Goa is split between the state-run Kadamba Transport Corporation (about 630 to 650 buses) and private operators (about 950 to 1,400 buses, around 70% of intra-state connectivity). Commuters have no reliable way to see where a bus is, official applications cover only KTCL buses, and dedicated GPS hardware costing ₹5,000 to ₹10,000 per bus is out of reach for small operators.

This project presents **Live Transit Tracking (Bus Tracker)**, a hardware-free, smartphone-native tracking system. The driver's Android phone streams its GPS position, through a foreground location service, to a Spring Boot server using STOMP over WebSocket. The server keeps the latest position of each active bus in memory and pushes updates to passengers using Server-Sent Events and STOMP topics. Passengers view KTCL and private buses on an OpenStreetMap map, filter by category and track a chosen bus live. Access for drivers is controlled by an administrator-managed phone whitelist, and an administrator web page manages schedules and e-mails generated credentials to drivers. Stale buses are removed automatically after 15 minutes without updates, and distance and ETA are computed with the Haversine formula.

The system was analysed and designed with UML (use case, class, sequence, activity, state machine, component and deployment diagrams), implemented with Java 21, Spring Boot 3.2.3, Kotlin and Jetpack Compose, persisted in an H2 database and packaged in a Docker container. Automated tests cover the whitelist and admin flows and route filtering. The report also documents the main limitations (prototype-level security, constant-speed ETA, single-node design) and a roadmap that includes authentication hardening, stop-level ETA, analytics and ticketing.

Keywords: public transport, live tracking, Android, Spring Boot, WebSocket, STOMP, Server-Sent Events, OpenStreetMap, Goa.

Table of Contents

Declaration i

Acknowledgements ii

Abstract / Executive Summary iii

Table of Contents iv

List of Figures vii

List of Tables viii

1. INTRODUCTION & PROJECT OVERVIEW 1

1.1 Domain & Background Context 1

1.2 Problem Statement 1

1.3 Project Objectives & Scope 2

1.3.1 Objectives 2

1.3.2 Target audience 2

1.3.3 Scope and boundaries 3

1.4 Expected Outcomes & Value Proposition 3

1.5 Report Structure 3

2. IDEATION, BRAINSTORMING & CONCEPT SELECTION 5

2.1 Need Assessment & Initial Problem Exploration 5

2.1.1 Market data 5

2.1.2 Competitor analysis 5

2.1.3 User pain points 5

2.2 Brainstorming Methodology 6

2.3 Evaluation of Proposed Ideas 6

2.3.1 Accepted Idea: Smartphone-based live tracking 6

2.3.2 Rejected Idea 1: Dedicated IoT GPS hardware 7

2.3.3 Rejected Idea 2: Ticketing and payments super-app 7

2.4 Decision Matrix & Comparative Analysis 7

3. FEASIBILITY & REQUIREMENT ANALYSIS 9

3.1 Feasibility Study 9

3.1.1 Technical Feasibility 9

3.1.2 Economic & Resource Feasibility 9

3.1.3 Operational & Legal Feasibility 10

3.2 Functional Requirements (FRs) 10

3.3 Non-Functional Requirements (NFRs) 11

3.4 System Dependencies & Hardware/Software Prerequisites 11

4. SYSTEM DESIGN & ARCHITECTURE 13

4.1 High-Level Architecture (HLD) 13

4.2 Data Flow Diagrams (DFDs) 14

4.2.1 Level 0 (Context diagram) 14

4.2.2 Level 1 DFD 15

4.2.3 Level 2 DFD (Process 3.0: Receive & Broadcast Location) 16

4.3 Unified Modeling Language (UML) Diagrams 16

4.3.1 Use Case Diagram & Actor Profiles 16
4.3.2 Sequence Diagrams (Core User Workflows) 17
4.3.3 Activity & State Machine Diagrams 19
4.3.4 Class / Component Diagrams 22
4.4 Database & Schema Design 25
4.4.1 Data dictionary 25
5. IMPLEMENTATION & TECHNOLOGY STACK 27
5.1 Technology Stack Rationale 27
5.2 Environment Setup & Development Pipeline 27
5.2.1 Repository layout 27
5.2.2 Setup steps 28
5.2.3 Version control and workflow 28
5.3 Module-Wise Implementation Breakdown 28
5.3.1 Module 1: Authentication & User Management 28
5.3.2 Module 2: Core Engine & Business Logic 29
5.3.3 Module 3: Administration Dashboard 30
5.4 Key Algorithms & Source Code Highlights 30
5.4.1 Receiving and broadcasting a location update 30
5.4.2 Distance and ETA 31
5.4.3 Automatic cleanup of inactive buses 31
5.4.4 Driver onboarding with generated credentials 31
5.4.5 SSE heartbeat 32
5.4.6 Route filtering on the driver app 32
6. WORKING PRODUCT & USER INTERFACE SCREENSHOTS 33
6.1 Application Walkthrough & Screenshots 33
6.1.1 Admin Interface 33
6.1.2 Onboarding, Login and Role Selection 34
6.1.3 Passenger Dashboard and Live Tracking 35
6.1.4 Driver Operational Workflow 37
6.2 Error Handling & Edge-Case UI Screens 40
7. TESTING, VERIFICATION & QUALITY ASSURANCE 42
7.1 Testing Strategy & Frameworks 42
7.2 Test Cases & Execution Results 42
7.2.1 Automated tests present in the project 42
7.2.2 Manual system test cases 43
7.3 User Acceptance Testing (UAT) 44
7.4 Performance & Security Analysis 45
7.4.1 Performance 45
7.4.2 Security analysis 45
8. PROJECT MANAGEMENT & COST ESTIMATION 47
8.1 Work Breakdown Structure (WBS) & Task Allocation 47
8.2 Project Timeline & Gantt Chart 47
8.3 Risk Management Matrix 48
8.4 Cost & Resource Estimation 49
8.4.1 Development costs 49

8.4.2 Infrastructure (running) costs 49
8.4.3 Business model (from project proposal) 49
9. CONCLUSION, CHALLENGES & FUTURE SCOPE 50
9.1 Summary of Deliverables 50
9.2 Major Challenges Encountered & Key Learnings 50
9.2.1 Technical challenges addressed in the design 50
9.2.2 Organisational learnings 51
9.3 Limitations of Current System 51
9.4 Future Enhancements & Scalability Roadmap 51
REFERENCES / BIBLIOGRAPHY 53
APPENDIX A: SOURCE CODE LISTINGS & CONFIGURATION FILES 54
APPENDIX B: API DOCUMENTATION & ENDPOINT SPECIFICATIONS 56
APPENDIX C: RAW SURVEY DATA & USER TESTING CHECKLISTS 58
C.1 Commuter survey questionnaire 58
C.2 User acceptance testing checklist 58
C.3 Satisfaction questionnaire (1 = strongly disagree, 5 = strongly agree) 59

List of Figures

Figure 4.1: High-level architecture of the Bus Tracker system 13
Figure 4.2: Deployment diagram 14
Figure 4.3: Level 0 DFD (context diagram) 15
Figure 4.4: Level 1 DFD 15
Figure 4.5: Level 2 DFD for process 3.0 16
Figure 4.6: Use case diagram 16
Figure 4.7: Sequence diagram: live tracking 18
Figure 4.8: Sequence diagram: administrator setup 19
Figure 4.9: Activity diagram: driver login and live location sharing 20
Figure 4.10: State machine diagram (1/2): bus 21
Figure 4.11: State machine diagram (2/2): passenger live-update connection 21
Figure 4.12: Class diagram (1/2): domain model 22
Figure 4.13: Class diagram (2/2): backend design 23
Figure 4.14: Component diagram 24
Figure 4.15: Entity-relationship diagram 25
Figure 6.1: Admin Login page (username, password, Login button) 33
Figure 6.2: Admin Driver Whitelist (add phone number, whitelisted numbers, Remove) 34
Figure 6.3: Passenger login and role selection (Passenger/Driver toggle) 35
Figure 6.4: Driver login with KTCL bus category and Get OTP 35
Figure 6.5: Passenger map view with bus details card 36
Figure 6.6: Active buses side panel (search, filters, Track Live) 36
Figure 6.7: Live tracking of a bus (Tracking Live status) 37
Figure 6.8: Driver login with Private bus category and OTP sent confirmation 38
Figure 6.9: Driver Console with validation warning for empty details 38
Figure 6.10: Driver route selection: Start Point dropdown 39
Figure 6.11: Driver route selection: Panaji to Advoi via Marcel-Sanquelim 39
Figure 6.12: Driver sharing live location (Broadcasting Active) 40
Figure 8.1: Gantt chart (planned schedule) 48

List of Tables

Table 1.1: Project scope 3
Table 1.2: Report organisation 3
Table 2.1: Market analysis of Goa public transport 5
Table 2.2: Competitor analysis 5
Table 2.3: Decision matrix 7
Table 3.1: Technology choices and feasibility 9
Table 3.2: Functional requirements 10
Table 3.3: Non-functional requirements 11
Table 3.4: System prerequisites 11
Table 4.1: Actor profiles 16
Table 4.2: Data dictionary: user_accounts 25
Table 4.3: Data dictionary: bus_drivers 25
Table 4.4: Data dictionary: driver_whitelist 26
Table 4.5: Data dictionary: bus_schedules 26
Table 4.6: Data dictionary: BusLocationUpdate (transient message, not persisted) 26
Table 5.1: Technology stack rationale 27
Table 6.1: Error and edge-case handling 40
Table 7.1: Testing levels 42
Table 7.2: Automated test inventory 42
Table 7.3: Manual system test cases 43
Table 7.4: UAT tasks 44
Table 7.5: UAT results summary 44
Table 7.6: Performance benchmarks 45
Table 7.7: Security analysis 45
Table 8.1: Work breakdown structure 47
Table 8.2: Project timeline 47
Table 8.3: Risk management matrix 48
Table 8.4: Development cost estimate 49
Table 8.5: Infrastructure cost estimate 49
Table 8.6: Proposed business model 49
Table 9.1: Objectives versus achievements 50
Table 9.2: Future roadmap 51
Table B.1: REST and WebSocket interface summary 56
Table C.1: UAT checklist 58

1. INTRODUCTION & PROJECT OVERVIEW

1.1 Domain & Background Context

Public road transport is the backbone of daily mobility for a large share of commuters in Indian towns and cities. In Goa, the bus network has a dual structure: the state-run Kadamba Transport Corporation Limited (KTCL) operates a fleet of roughly 630 to 650 buses, while private stage-carriage operators run an estimated 950 to 1,400 buses that provide about 70% of intra-state connectivity. Official estimates put the daily bus commuter volume between 150,000 and 250,000 passengers.

Despite this scale, most commuters still depend on printed timetables, word of mouth or simple waiting at the roadside. The Goa Comprehensive Mobility Plan reports that buses account for only 11.4% of daily trips compared with 59.0% for two-wheelers, and transit analyses report wait times of 20 to 45 minutes or more in off-peak hours, with very few services after 7 to 8 PM. A major reason commuters prefer private vehicles is the uncertainty of public transport: they cannot tell whether a bus is a minute away or has already left.

Real-time vehicle tracking is the standard technological answer to this uncertainty. Conventional implementations, however, require a dedicated GPS unit installed in every bus. Under the Central Motor Vehicles Rules (AIS-140), such hardware costs approximately ₹5,000 to ₹10,000 per bus, plus connectivity and maintenance. Small private operators, who own one or a few buses, find this cost hard to justify, and as a result the majority of Goa's bus fleet remains invisible to digital tools.

Modern Android smartphones already contain an accurate GPS receiver, mobile data connectivity and a capable processor. This project exploits that observation: the driver's own phone is used as the tracking device, and a lightweight server relays the position to commuters in real time. The system is called **Live Transit Tracking (Bus Tracker)**.

1.2 Problem Statement

Commuters in Goa have no single, reliable way to see where a bus is right now. The specific problems identified are:

- **Partial coverage.** The official KTCL application tracks only state-run buses, which leaves the roughly 70% of intra-state connectivity provided by private operators unrepresented.

- **Static information.** General-purpose map applications provide timetables for major routes but no real-time positions in Goa, so delays and cancelled trips are invisible to the passenger.
- **Hardware barrier.** Dedicated GPS hardware is a recurring cost that small private operators are unlikely to bear.
- **Unreliable pings and feature bloat.** Existing applications are reported to lose location updates in weak-network corridors, and super-apps dilute the transit experience with unrelated services.
- **No operator control.** There is no lightweight way for an operator to register a bus, a route and a driver so that only authorised drivers can broadcast a position.

The problem to be solved is therefore: **design and build a hardware-free, smartphone-native system that lets authorised drivers share their live location and lets any commuter view and track KTCL and private buses on a map, with operator-side administration of drivers and schedules.**

1.3 Project Objectives & Scope

1.3.1 Objectives

- Use the built-in GPS of the driver's Android phone as the only tracking hardware.
- Stream bus positions to passenger devices with low latency using WebSocket (STOMP) for ingestion and Server-Sent Events (SSE) for delivery.
- Let passengers view all buses on an OpenStreetMap-based map, filter them by operator category (KTCL or Private), and track a chosen bus live.
- Restrict broadcasting to drivers whose phone numbers have been whitelisted by an administrator.
- Provide an administrator web page for managing the phone whitelist and creating, editing and deleting bus schedules, and automatically e-mail credentials to drivers.
- Provide an estimated time of arrival (ETA) from distance and an average speed.
- Apply object-oriented analysis and design throughout and document it with UML diagrams.

1.3.2 Target audience

Three user groups are served: **passengers** (daily commuters, students, tourists), **drivers** of KTCL and private buses, and **administrators** (operators or transport staff) who register drivers and schedules.

1.3.3 Scope and boundaries

Table 1.1: Project scope

In scope	Out of scope (current version)
Android application with passenger and driver modes	iOS application
Live location sharing and tracking, map view, category filter	Ticketing, payments and smart-card integration
Spring Boot backend with REST, STOMP and SSE interfaces	Machine-learning-based ETA prediction
Driver whitelist, schedule management, e-mail of credentials	Production-grade identity (JWT, SMS OTP, hashed passwords)
Docker-based deployment of the backend	Multi-server scaling and a persistent location history store

1.4 Expected Outcomes & Value Proposition

The key deliverables of the project are:

- A working Android application (package `com.project.bus_tracker`) with passenger and driver modes.
- A containerised Spring Boot backend (`bus-tracker-web`, version 2.0.0) with an administrator web page and an API documentation page.
- A complete set of UML models: use case, class, sequence, activity, state machine, component and deployment diagrams.
- This project report, together with the source code and automated tests.

The value proposition is a **zero-hardware-cost** tracking system. For an operator with 100 buses, the avoided hardware spend at ₹5,000 to ₹10,000 per bus is ₹5 lakh to ₹10 lakh. For commuters, the system reduces waiting uncertainty and covers private buses that official applications ignore. For authorities, it provides a first digital view of an otherwise unmonitored part of the network.

1.5 Report Structure

Table 1.2: Report organisation

Chapter	Content
1 Introduction	Domain, problem, objectives, scope, outcomes
2 Ideation & Concept Selection	Need assessment, brainstorming, accepted and rejected ideas, decision matrix
3 Feasibility & Requirements	Technical, economic, operational and legal feasibility; FRs, NFRs, prerequisites

Chapter	Content
4 System Design & Architecture	HLD, DFDs, UML diagrams, database design and data dictionary
5 Implementation & Technology Stack	Stack rationale, environment, module-wise implementation, key algorithms
6 Working Product & UI	Wireframes, application walkthrough, error and edge-case screens
7 Testing & QA	Strategy, test cases, UAT, performance and security analysis
8 Project Management & Cost	WBS, timeline, risk matrix, cost estimate
9 Conclusion & Future Scope	Achievements, challenges, limitations, roadmap
References & Appendices	Citations, code listings, API specification, survey and testing checklists

2. IDEATION, BRAINSTORMING & CONCEPT SELECTION

2.1 Need Assessment & Initial Problem Exploration

The need assessment began with the transport landscape of Goa and an examination of existing applications. Two sources of evidence were used: published statistics on ridership, fleet size and delays, and a feature-level comparison of competing applications.

2.1.1 Market data

Table 2.1: Market analysis of Goa public transport

Parameter	Data point / statistic	Source reference
Daily bus commuter volume	150,000 to 250,000 passengers per day	The Times of India (CM statement)
Private stage-carriage fleet	~950 to 1,400 buses (about 70% of intra-state connectivity)	Goa Transport Dept / DTE report
KTCL state fleet size	~630 to 651 buses (460 diesel + 171 electric AC)	KTC official filings / TOI
Public transit modal split	11.4% bus vs 59.0% two-wheeler daily trips	Goa Comprehensive Mobility Plan (CMP)
Off-peak service and delays	20 to 45+ minute waits; scarce services after 7 to 8 PM	Down To Earth transit analysis
Hardware GPS deployment cost	₹5,000 to ₹10,000 per bus (AIS-140 compliance)	Central Motor Vehicles Rules
Mhaji Bus scheme subsidy	₹3 per km operational subsidy + ₹10 lakh replacement grant	Goa Transport Policy notification

2.1.2 Competitor analysis

Table 2.2: Competitor analysis

Application	Features available	Status	Gaps
Goa On The Go (official KTCL app)	Live KTC fleet tracking, QR tickets, KTC Smart Transit Card recharge	Partially working	No private buses (~70% of market); unreliable pings in weak cellular corridors; no multi-modal integration
MajheApp (KTCL / GraceFulGoa super-app)	QR ticketing, live tracking, pilot bike-taxi booking	Active rollout	Private operators virtually absent; feature bloat dilutes transit UX; no crowd-sourced location data
Google Maps	Static route planning and timetables for major KTC routes	Static only	No real-time GPS in Goa; static timetables fail under delays; private routes missing
Chalo App	Live GPS, ETA, occupancy, mobile passes (in other cities)	Non-functional in Goa	No local Goa coverage; no integration with Goan private operators

2.1.3 User pain points

- Commuters cannot tell whether a bus has already passed or is delayed.

- Private bus users are not served by any live tracking application.
- Drivers and operators do not want to buy and maintain extra devices.
- Operators lack a simple tool to register buses and drivers.

2.2 Brainstorming Methodology

Ideation followed a structured sequence of divergent and convergent steps.

Problem framing. The team restated the problem from the commuter's point of view ("Where is my bus now?") and the operator's point of view ("How do I make my bus visible cheaply?").

- **Mind mapping.** A mind map with "Live Transit Tracking" at the centre was expanded into branches: data source (hardware, phone, passengers), delivery (polling, push), users (passenger, driver, admin), business (subscription, ads) and constraints (cost, privacy, network).
- **Idea listing and clustering.** Candidate solutions were listed without judgement and then clustered into three concepts: smartphone-based tracking, dedicated hardware tracking and an all-in-one ticketing super-app.
- **Evaluation.** Each concept was assessed against cost, time-to-build, accuracy, scalability, adoption potential and fit with the course (object-oriented technology), and the results were summarised in a weighted decision matrix (Section 2.4).

2.3 Evaluation of Proposed Ideas

2.3.1 Accepted Idea: Smartphone-based live tracking

Concept. The driver logs in to the Android application, chooses a route and taps Start. A foreground service reads GPS fixes through Google Play Services Fused Location and sends them over a STOMP WebSocket to the server. The server stores the latest position of each bus in memory and pushes it to every subscribed passenger over SSE and to STOMP topics. Passengers see buses as markers on an OpenStreetMap map and can track any bus.

Justification.

- No hardware purchase: the driver's existing phone is the device, which removes the main adoption barrier for private operators.
- Uses mature, free technology (Android, Spring Boot, OpenStreetMap) that the team already has the skills to use.
- Rich object model (accounts, drivers, schedules, location updates, sessions) that suits an object-oriented technology project.
- Scales naturally: adding a bus adds one more phone.

Alignment with goals. The idea satisfies every objective in Section 1.3.1: hardware-free tracking, low-latency delivery, category filtering, whitelist-based driver authorisation and administrator tooling.

2.3.2 Rejected Idea 1: Dedicated IoT GPS hardware

Concept. Install an AIS-140 compliant GPS tracker in every bus that reports to a cloud platform, from which a passenger application reads positions.

Bottlenecks. Each unit costs ₹5,000 to ₹10,000, plus SIM data and installation, which is a financial barrier for single-bus private operators. Procurement, installation and field maintenance cannot be done within a semester project, and testing would depend on physical access to vehicles. The device would also provide no built-in link to driver authentication.

Reason for rejection. Cost and logistics defeat the central goal of an inclusive system that covers private buses; the smartphone provides the same data at no extra hardware cost.

2.3.3 Rejected Idea 2: Ticketing and payments super-app

Concept. Build a combined application with live tracking, QR ticket purchase, wallet and smart-card recharge, similar to current official applications.

Complexity and scope constraints. Payment gateways, settlement, refunds, fraud handling and integration with the KTC smart card require commercial agreements, regulatory compliance and security hardening far beyond the available time. Competitor analysis also shows that feature bloat dilutes the transit experience.

Trade-off analysis. A super-app would offer more features but at the cost of focus, schedule risk and legal exposure. A focused tracking product delivers the most commuter value per unit of effort. Ticketing is retained as a future extension (Chapter 9).

2.4 Decision Matrix & Comparative Analysis

Each concept was scored from 1 (poor) to 5 (excellent) against weighted criteria. The weighted score is the sum of weight × score.

Table 2.3: Decision matrix

Criterion	Weight	Smartphone tracking	Dedicated IoT hardware	Ticketing super-app
Low cost to adopt	25%	5	1	3
Feasibility within project time	20%	5	2	2
Coverage of private buses	20%	5	3	2
Location accuracy and reliability	15%	4	5	3

Criterion	Weight	Smartphone tracking	Dedicated IoT hardware	Ticketing super-app
Scalability	10%	4	3	3
Fit with object-oriented technology course	10%	5	2	4
Weighted score (out of 5)	100%	4.75	2.50	2.70

The smartphone-based concept scores highest and was selected. Its main weakness, dependence on the driver's phone battery, network and cooperation, is addressed by a foreground service, inactivity cleanup and administrator-controlled onboarding.

3. FEASIBILITY & REQUIREMENT ANALYSIS

3.1 Feasibility Study

3.1.1 Technical Feasibility

The selected stack is mainstream and well documented. All components were confirmed to work together in the implementation.

Table 3.1: Technology choices and feasibility

Area	Choice	Assessment
Backend	Java 21, Spring Boot 3.2.3 (Web, WebSocket, Data JPA, Security, Mail)	Team knows Java from the course; Spring provides REST, STOMP and SSE support out of the box
Database	H2 file database (`./data/bustracker`)	No separate server needed; adequate for prototype scale
Mobile	Kotlin, Jetpack Compose, Material 3; minSdk 24, targetSdk 36	Covers the vast majority of active Android devices
Maps	osmdroid 6.1.20 with OpenStreetMap tiles	Free, no API key or billing account
Networking	OkHttp 4.12.0 with okhttp-sse; STOMP client over WebSocket	Reliable streaming on mobile networks
Location	Google Play Services Location 21.3.0 (Fused Location)	Accurate, battery-aware GPS
Deployment	Docker multi-stage build (Maven, eclipse-temurin 21 JRE Alpine)	Reproducible, runs on any container host

Skill readiness: the team had prior experience with Java, object-oriented design and Android basics; Kotlin Compose and Spring WebSocket were learned during the project. Infrastructure needed is a laptop with JDK 21 and Android Studio, an Android phone for testing and a container-capable host for the server.

3.1.2 Economic & Resource Feasibility

All development tools and libraries are free and open source (Spring Boot, Android Studio, OpenStreetMap, osmdroid, H2, Docker Engine). There is no hardware cost for tracking. Recurring costs are limited to server hosting and an SMTP account for credential e-mails. A detailed estimate is given in Section 8.4.

Compared with hardware tracking at ₹5,000 to ₹10,000 per bus, the approach is economically attractive. The proposed business model (Section 8.4.3) suggests a subscription of ₹50 to ₹90 per bus per month for operators, a free ad-supported passenger tier, a ₹20 per month premium tier and route-based local advertising.

3.1.3 Operational & Legal Feasibility

- **Adoption.** Drivers must keep the application running with location enabled. The foreground-service notification and a simple Start/Stop flow keep this easy. Operators must agree to register their buses.
- **Privacy.** Only the location of a bus, shared voluntarily by its driver while on duty, is exposed. Passenger locations are not collected. Sharing stops when the driver presses Stop or after 15 minutes without an update.
- **Compliance.** Applicable Indian data protection principles (consent, purpose limitation, minimal data) should be followed. Phone numbers and e-mail addresses are personal data and must be protected. The system supplements, and does not replace, any AIS-140 obligation on buses that legally require it.
- **Map data licensing.** OpenStreetMap tiles are used under the OSM usage policy with attribution; heavy production use would require a commercial or self-hosted tile provider.

3.2 Functional Requirements (FRs)

Table 3.2: Functional requirements

ID	Requirement	Actor
FR-01	The system shall allow a passenger to sign up with username, password and display name, and to log in and out.	Passenger
FR-02	The system shall allow a driver to log in with a whitelisted phone number, bus category and a one-time code, and shall reject non-whitelisted numbers with an error.	Driver
FR-03	The system shall allow a driver to select route, start point and destination and to start sharing live GPS location.	Driver
FR-04	The system shall accept location updates over STOMP (`/app/update-location`) and store the latest position per bus.	System
FR-05	The system shall broadcast each update to SSE subscribers and to STOMP topics (`/topic/bus/{busId}`, `/topic/bus-locations`, `/topic/active-buses`).	System
FR-06	The system shall allow a driver to stop sharing, remove the bus from the active set, notify clients and reset the marker to the scheduled start position.	Driver
FR-07	The system shall automatically remove buses that have sent no update for 15 minutes (cleanup every 60 seconds).	System
FR-08	The system shall display all active and scheduled buses on an OpenStreetMap map and allow filtering by KTCL or Private.	Passenger
FR-09	The system shall allow a passenger to track and untrack one or more buses and receive live events for them.	Passenger
FR-10	The system shall maintain SSE connections with a heartbeat every 15 seconds.	System
FR-11	The system shall provide distance (Haversine) and ETA calculation for a bus.	System

ID	Requirement	Actor
FR-12	The system shall provide an administrator login protected by credentials and a session.	Admin
FR-13	The system shall allow the administrator to add, list and remove whitelisted driver phone numbers.	Admin
FR-14	The system shall allow the administrator to create, edit and delete bus schedules.	Admin
FR-15	On schedule creation the system shall generate a six-digit driver password, store the driver and e-mail the Bus ID and password to the driver.	Admin, SMTP
FR-16	The system shall publish an API documentation page at `/docs`.	All

3.3 Non-Functional Requirements (NFRs)

Table 3.3: Non-functional requirements

Category	Requirement / target
Performance	A location update should reach connected passengers within about 1 to 2 seconds under normal network conditions. REST responses should complete within 500 ms for typical loads.
Scalability	Stateless REST layer and in-memory maps keyed by bus ID support hundreds of concurrent buses on one node; horizontal scaling needs an external broker (future work).
Availability	Server runs in a Docker container with restart capability; SSE clients reconnect after drop; accounts and schedules persist in H2 across restarts.
Security	Driver access restricted by whitelist; whitelist endpoints protected by an admin session; driver credentials generated randomly and delivered by e-mail. Known gaps (plain-text passwords, open security filter) are listed in Section 9.3.
Usability	Simple three-role interface, map-first home screen, minimal taps to start or stop sharing, category filter, side panel of active buses.
Reliability	Stale buses are cleaned automatically; heartbeat keeps connections alive; foreground service keeps GPS running while the screen is off.
Portability	Backend runs on any host with JDK 21 or Docker; Android app supports API level 24 and above.
Maintainability	Layered code (controllers, services, repositories, entities); automated tests for backend and route filtering.

3.4 System Dependencies & Hardware/Software Prerequisites

Table 3.4: System prerequisites

Component	Minimum requirement
Server	JDK 21 (or Docker), Maven 3.6+, 1 vCPU, 1 GB RAM, port 8080 open, outbound SMTP access
Database	Embedded H2 (no installation); writable `./data` directory
Developer machine	Android Studio with Kotlin 2.2.x and Compose plugin, Gradle wrapper, Git

Component	Minimum requirement
Driver phone	Android 7.0 (API 24) or newer, GPS, mobile data, Google Play Services, location permission granted
Passenger phone	Android 7.0 (API 24) or newer, mobile data
Admin computer	Any modern web browser
External services	SMTP server (for example Gmail with app password), OpenStreetMap tile servers
Key libraries	Spring Boot 3.2.3, osmdroid 6.1.20, OkHttp 4.12.0 / okhttp-sse, play-services-location 21.3.0, Jetpack Compose BOM

4. SYSTEM DESIGN & ARCHITECTURE

4.1 High-Level Architecture (HLD)

The system follows a three-tier client-server architecture with a real-time messaging layer. Android clients (passenger and driver modes) and an administrator web page form the client tier. A single Spring Boot application forms the application tier and exposes three kinds of interface: REST endpoints for request-response operations, a STOMP message broker over WebSocket for receiving driver location updates, and Server-Sent Events for pushing live updates to passengers. An H2 file database holds persistent data (accounts, drivers, whitelist and schedules), whereas the latest position of each active bus is held in a concurrent in-memory map.

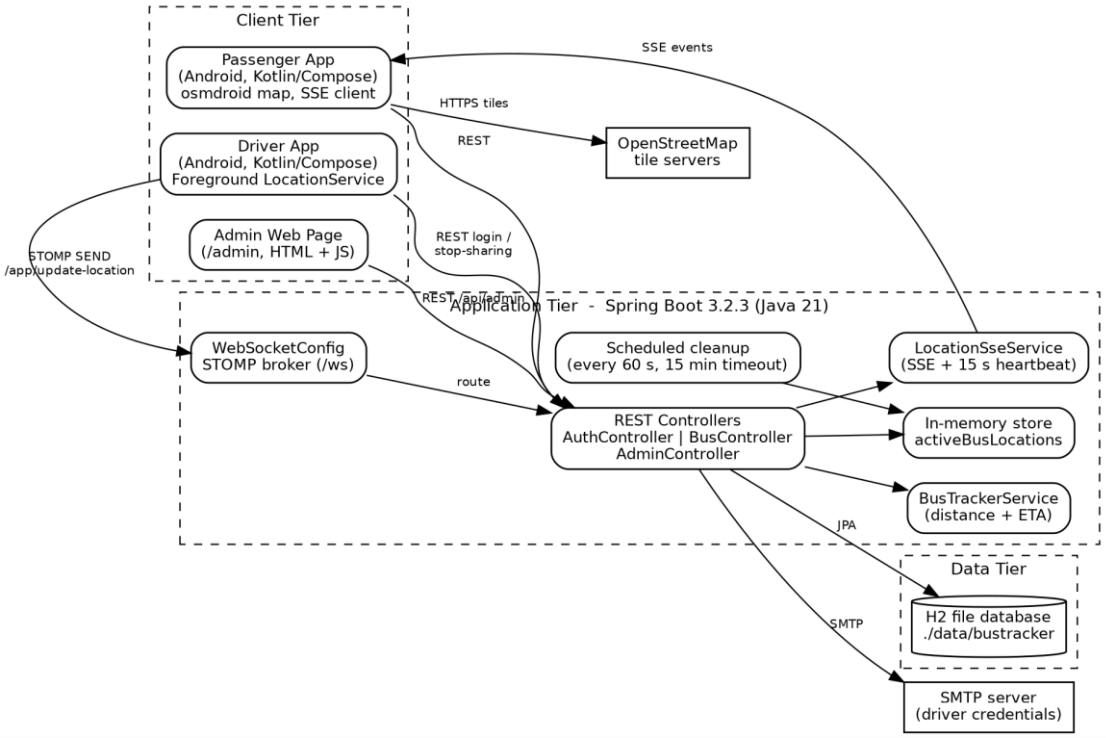


Figure 4.1: High-level architecture of the Bus Tracker system

The main modules and their relationships are:

- **Authentication module** (`AuthController`): signup, login, logout and OTP request; checks the driver whitelist.
- **Core tracking engine** (`BusController`, `LocationSseService`, `BusTrackerService`): receives GPS updates, keeps the active-bus map, broadcasts events, computes distance and ETA and runs the cleanup job.

- **Administration module** (`AdminController`, schedule endpoints in `BusController`, `admin/index.html`): whitelist and schedule management, credential e-mail.
- **Messaging configuration** (`WebSocketConfig`): STOMP endpoint `/ws` with SockJS fallback, `/app` application prefix and `/topic` simple broker.
- **Android client**: UI screens, `LocationService`, `StompWebSocketClient`, SSE client, `PassengerMapView`, repositories for cache, driver options and user session.

The physical deployment is shown in Figure 4.2. The backend runs as a Docker container (port 8080) on a server host; the driver and passenger applications run on Android phones; the administrator uses a web browser; and the server talks to an SMTP server and phones fetch map tiles from OpenStreetMap.

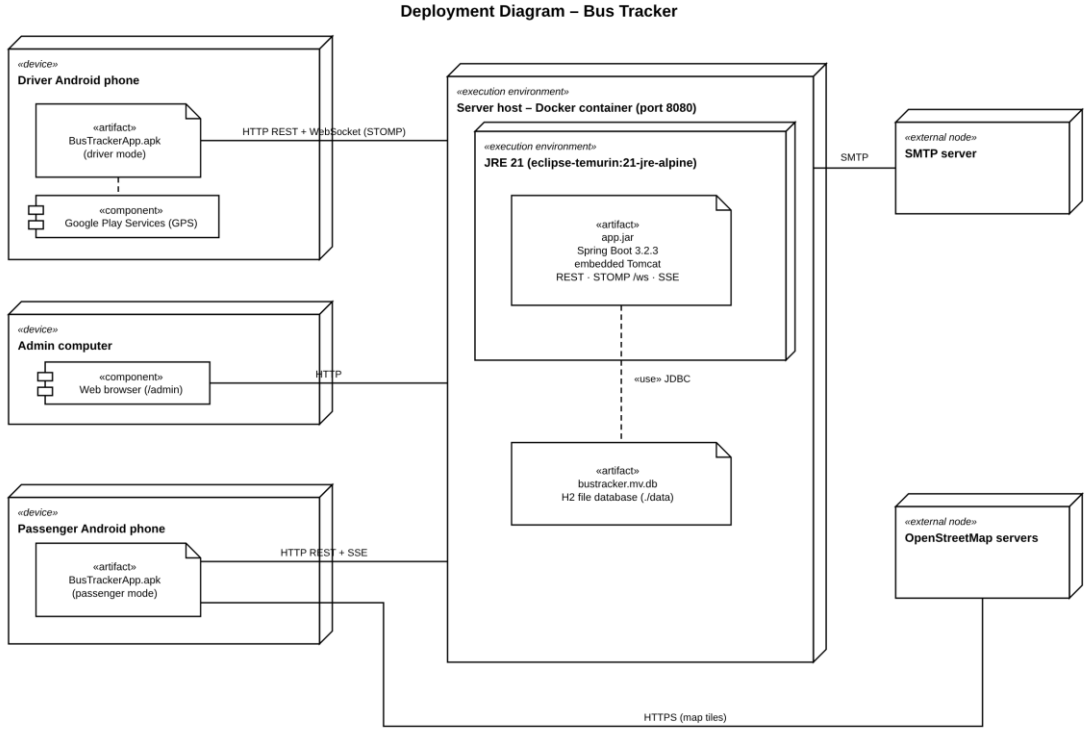


Figure 4.2: Deployment diagram

4.2 Data Flow Diagrams (DFDs)

4.2.1 Level 0 (Context diagram)

The context diagram treats the whole system as one process that exchanges data with four external entities: passengers, drivers, administrators and the SMTP mail server (plus the OpenStreetMap tile server that supplies map tiles directly to the passenger application).

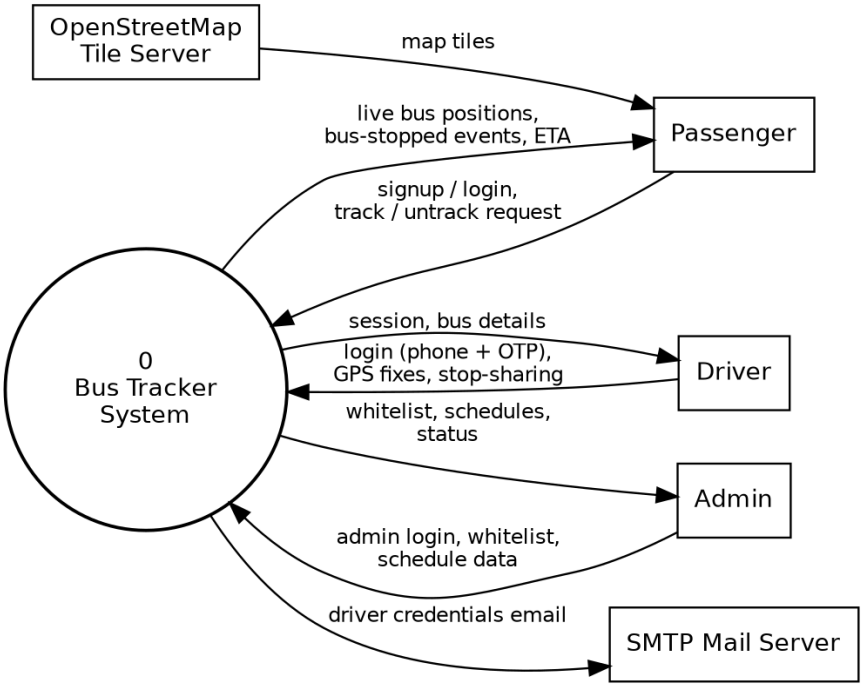


Figure 4.3: Level 0 DFD (context diagram)

4.2.2 Level 1 DFD

The system is decomposed into five processes: authentication (1.0), whitelist and schedule management (2.0), location receipt and broadcast (3.0), live tracking service over SSE (4.0) and inactive-bus cleanup (5.0). Four data stores are used: user and driver tables (D1), the driver whitelist (D2), bus schedules (D3) and the in-memory active-bus map (D4).

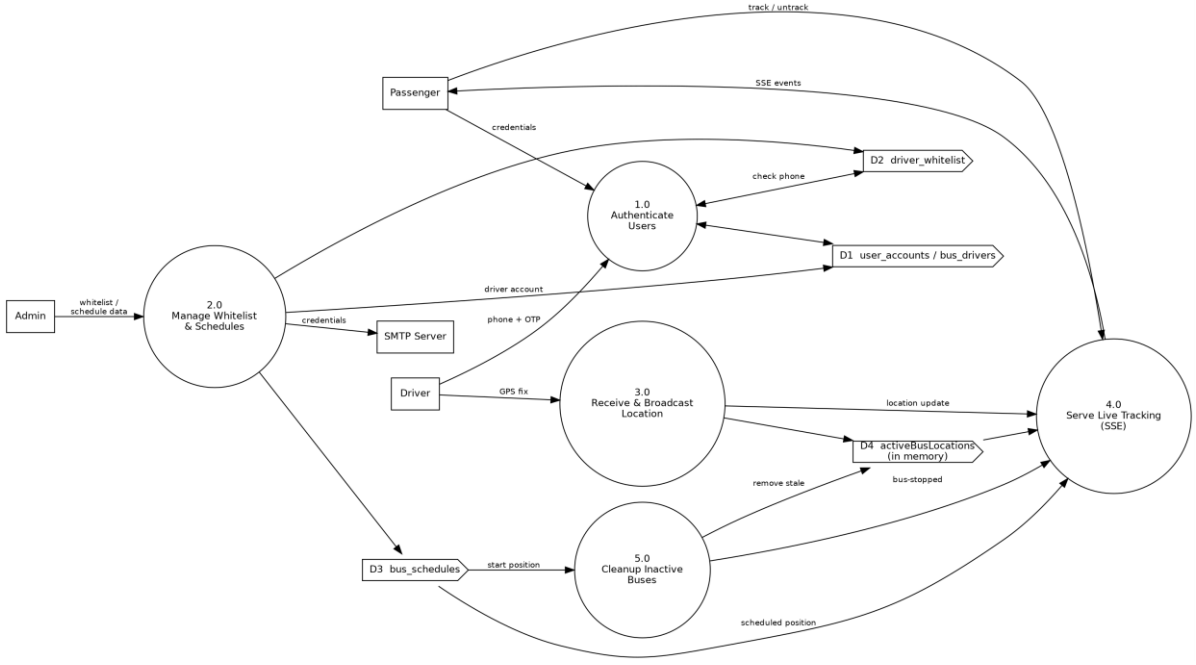


Figure 4.4: Level 1 DFD

4.2.3 Level 2 DFD (Process 3.0: Receive & Broadcast Location)

Process 3.0 is the heart of the system. A location update from the driver is validated (a bus ID must be present, otherwise the message is ignored), stored with a timestamp, the bus is registered as active, and the update is published to both delivery channels.

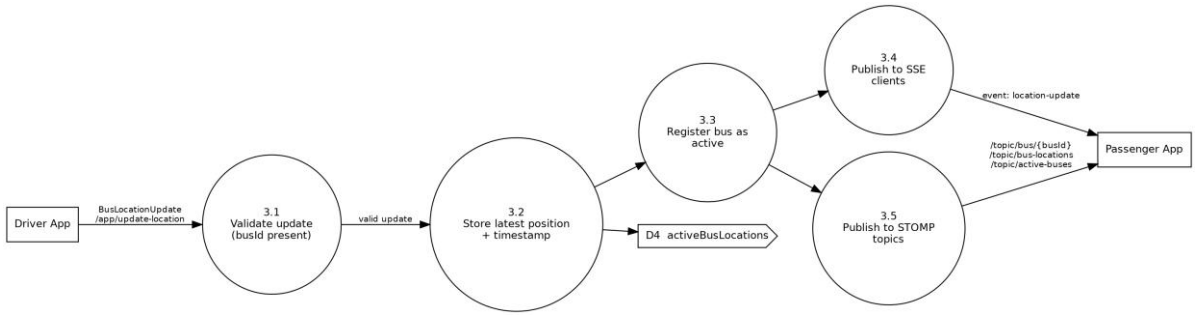


Figure 4.5: Level 2 DFD for process 3.0

4.3 Unified Modeling Language (UML) Diagrams

4.3.1 Use Case Diagram & Actor Profiles

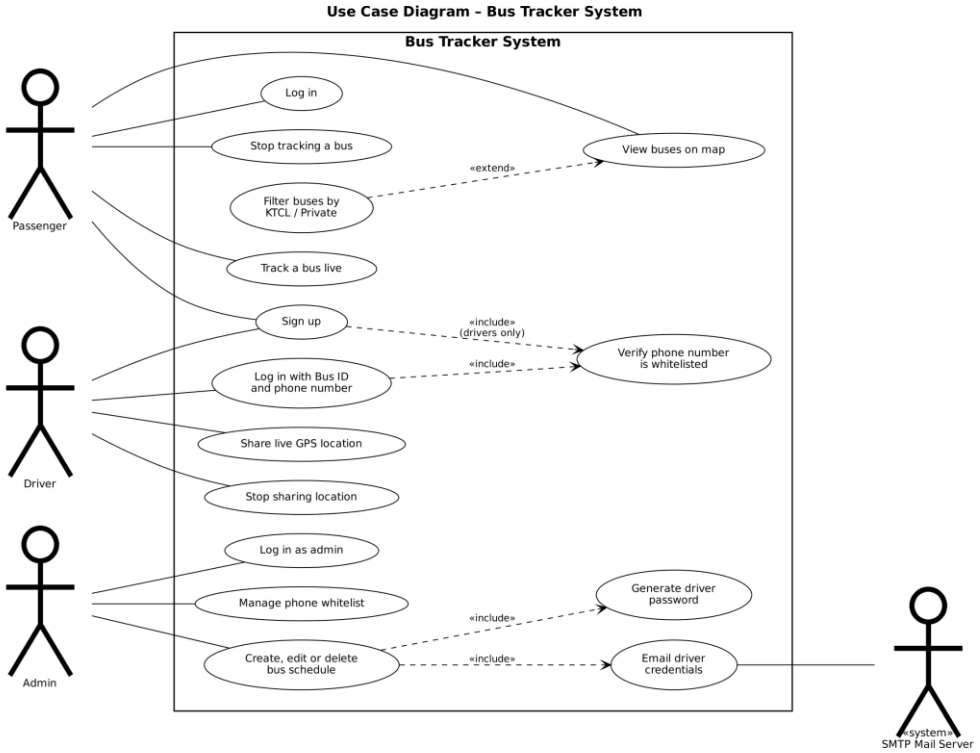


Figure 4.6: Use case diagram

Table 4.1: Actor profiles

Actor	Profile	Main use cases
Passenger	Commuter using the Android app in passenger mode; registers with a username and password.	Sign up, log in, view buses on map, filter by KTCL / Private (extends view), track a bus live, stop tracking a bus

Actor	Profile	Main use cases
Driver	KTCL or private bus driver whose phone number has been whitelisted; uses the app in driver mode.	Sign up and log in (both include whitelist verification), share live GPS location, stop sharing location
Admin	Operator or staff member using the web page at `/admin`.	Log in as admin, manage phone whitelist, create / edit / delete bus schedule (includes generating driver password and e-mailing credentials)
SMTP Mail Server (system actor)	External system that delivers e-mail.	Email driver credentials

Two relationships deserve mention. “Filter buses by KTCL / Private” extends “View buses on map” because filtering is an optional refinement. “Verify phone number is whitelisted” is included by both driver sign-up and driver login, and “Generate driver password” and “Email driver credentials” are included by schedule creation.

4.3.2 Sequence Diagrams (Core User Workflows)

Figure 4.7 shows the end-to-end live tracking workflow in four phases: (1) driver logs in; (2) a passenger opens the map, loads scheduled and active positions, opens the SSE stream and taps a bus to track it; (3) the driver shares location and, for every GPS fix, the server stores, broadcasts over SSE and publishes to STOMP topics, and the passenger’s marker moves; (4) the driver stops sharing, the server removes the bus, broadcasts a bus-stopped event and resets the marker to the scheduled start position.

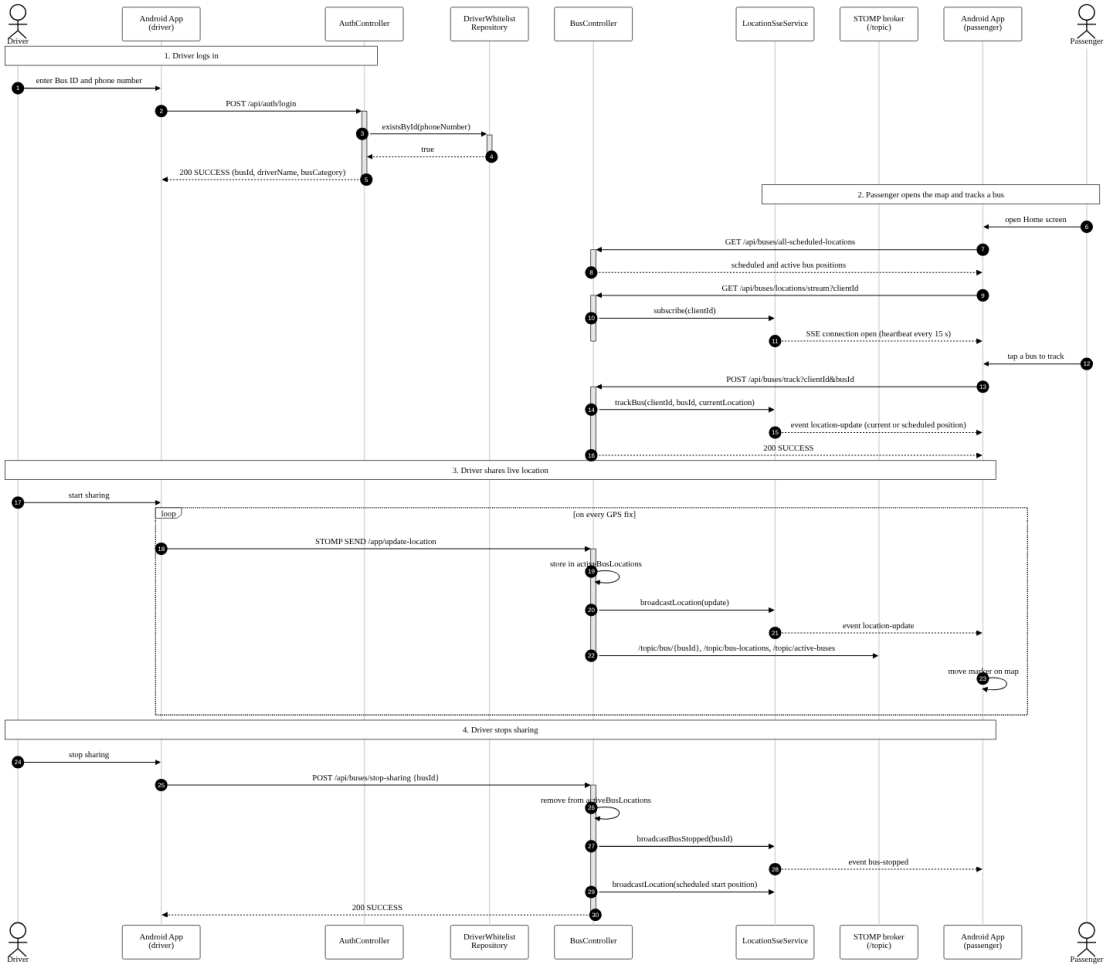


Figure 4.7: Sequence diagram: live tracking

Figure 4.8 shows the administrator workflow: the admin logs in, whitelists a driver phone number, and submits a schedule form. The server validates that the bus ID and driver e-mail are present (otherwise returns HTTP 400), generates a six-digit password, saves the schedule and the driver account, e-mails the credentials and publishes `/topic/schedule-added` and `/topic/bus-locations`.

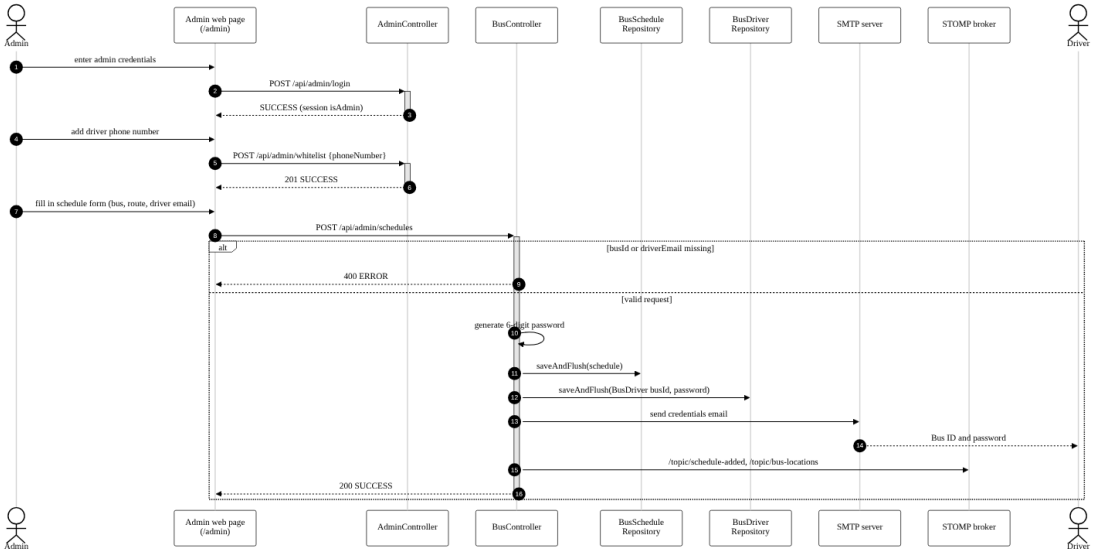


Figure 4.8: Sequence diagram: administrator setup

4.3.3 Activity & State Machine Diagrams

The activity diagram in Figure 4.9 uses three swimlanes (Driver, Android App, Spring Boot Server) to show login and location sharing, including the whitelist decision (HTTP 403 and retry), the location-permission check, starting the foreground service, connecting STOMP, the GPS loop and the stop path. A note records that the cleanup job performs the same removal after 15 minutes without an update.

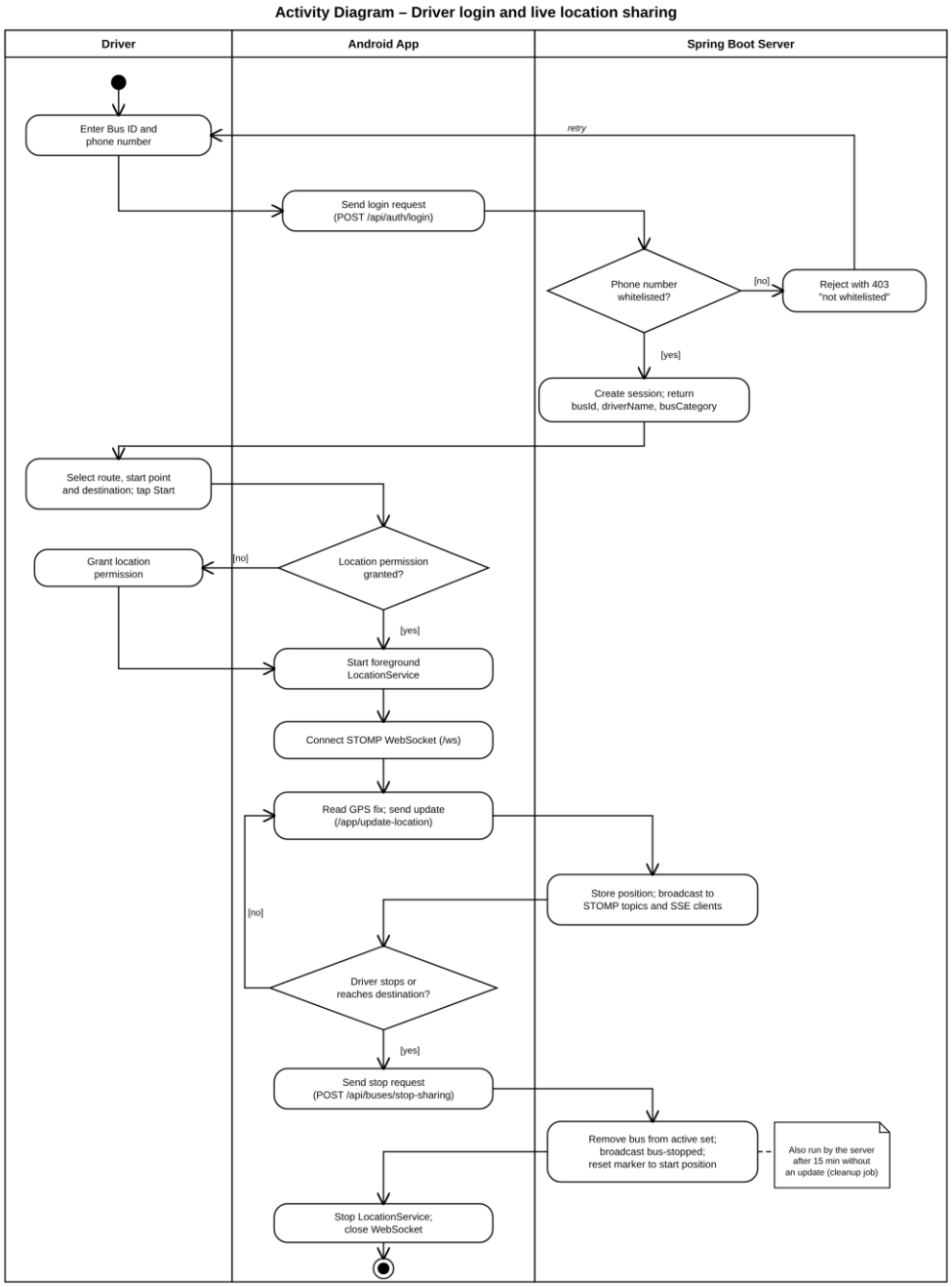


Figure 4.9: Activity diagram: driver login and live location sharing

The state machine in Figure 4.10 describes the life cycle of a bus. A bus is **Scheduled** when the admin creates a schedule (its marker is shown at the scheduled start position). It becomes **Active** when the first location update arrives and stays Active while updates keep refreshing `lastUpdated`. It becomes **Stopped** on a stop-sharing request or after 15 minutes without update (cleanup job every 60 seconds); on entry the bus is removed from the active set and a bus-stopped event is broadcast. From Stopped, the bus returns to Scheduled if a schedule still exists (broadcasting its start position) or ends if there is none. Deleting the schedule also ends the life cycle.

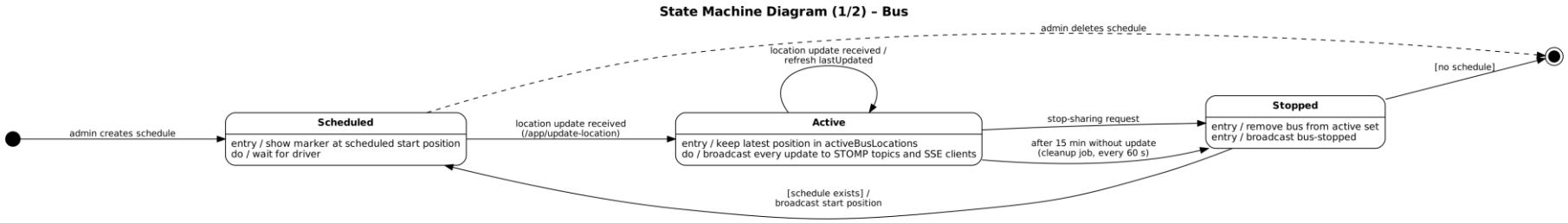


Figure 4.10: State machine diagram (1/2): bus

Figure 4.11 shows the passenger's live-update connection. A GET on `/api/buses/locations/stream` moves it from the initial state to **Subscribed**, where an SseEmitter and ClientSession are created and a heartbeat is sent every 15 s. A POST on `/api/buses/track` moves it to **Tracking**, where current or scheduled positions are sent and location-update and bus-stopped events are pushed for tracked buses. Untracking the last bus, a timeout or an error leads to **Closed**, where the ClientSession is removed.

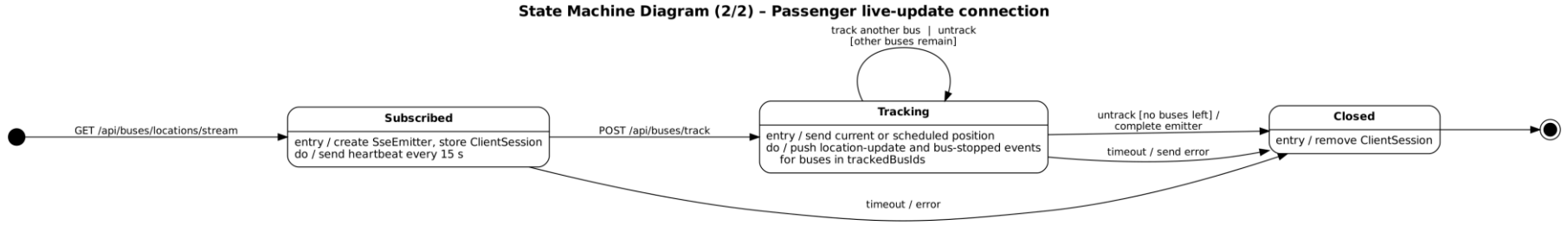


Figure 4.11: State machine diagram (2/2): passenger live-update connection

4.3.4 Class / Component Diagrams

The domain model in Figure 4.12 contains four entities and two DTOs. `UserAccount` (all users) and `BusDriver` (drivers, keyed by bus ID) both relate logically to `DriverWhitelist` through the phone number; `BusSchedule` relates to `BusDriver` through `busId`; and a schedule is used to instantiate a `BusLocationUpdate` (start position). The associations are logical: the entities do not declare JPA relationships and rows are matched by value in code. `PassengerBusStatus` is defined for a future per-passenger ETA view and is not used yet.

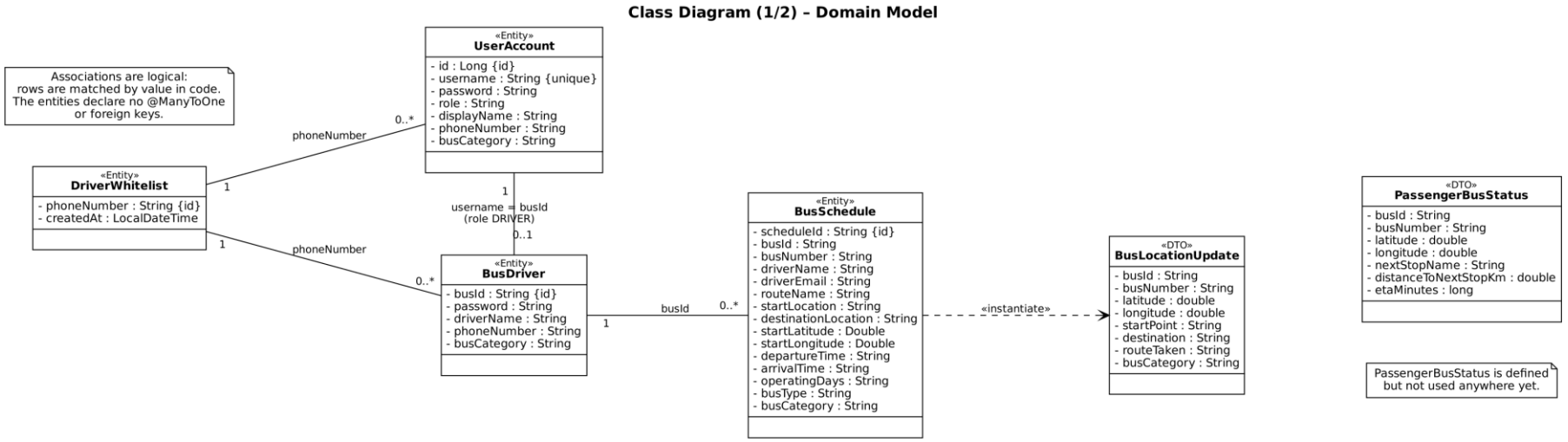


Figure 4.12: Class diagram (1/2): domain model

The backend design in Figure 4.13 shows the controllers (`AuthController`, `AdminController`, `BusController`), the services (`BusTrackerService`, `LocationSseService` with its composed `ClientSession` objects), the inner class `BusLocationWithTimestamp` and the Spring Data repositories. `BusController` delegates to `SimpMessagingTemplate` for STOMP and `JavaMailSender` for e-mail.

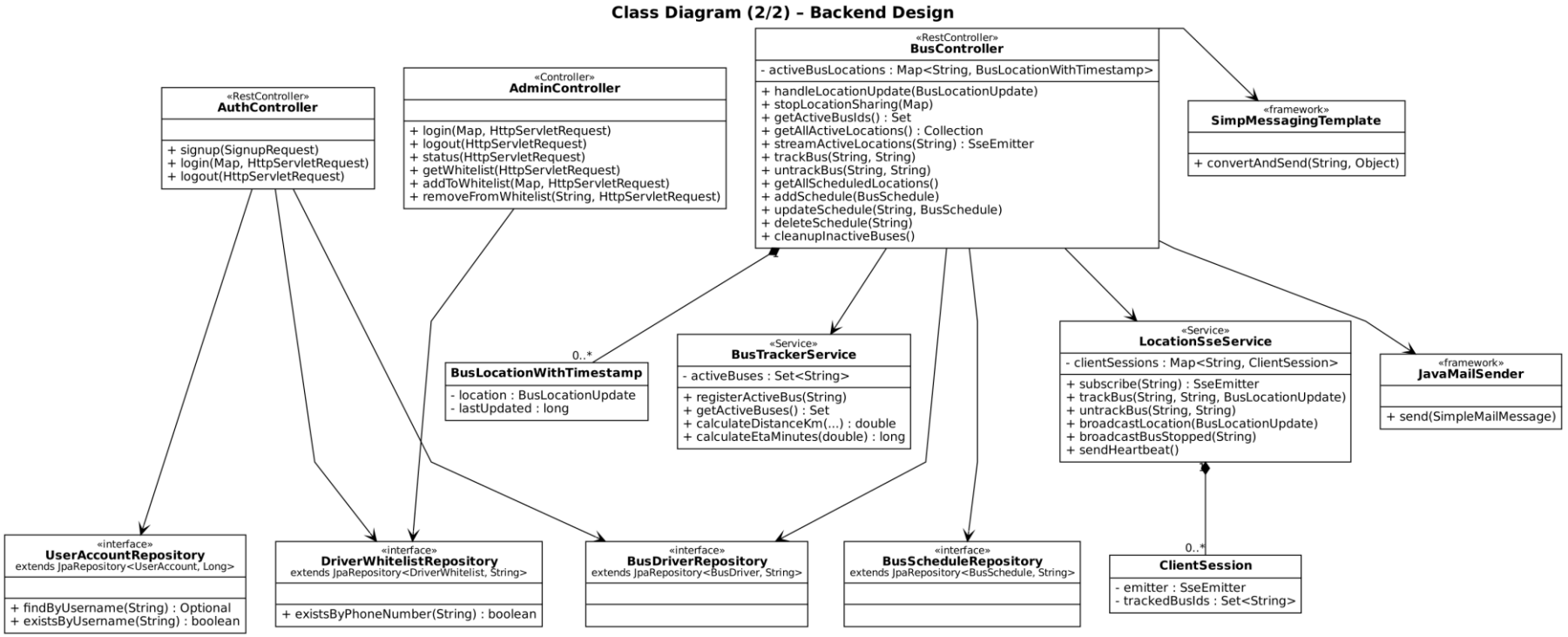


Figure 4.13: Class diagram (2/2): backend design

The component diagram in Figure 4.14 maps the Android components (UI screens, LocationService, PassengerMapView, StompWebSocketClient, SSE client, ApiConfig and repositories) to the backend components through REST (`/api/auth`, `/api/buses`, `/api/admin`), STOMP (`/ws`) and SSE, and shows the external services: Google Play Services Fused Location, OpenStreetMap tile server, SMTP server and the H2 file database.

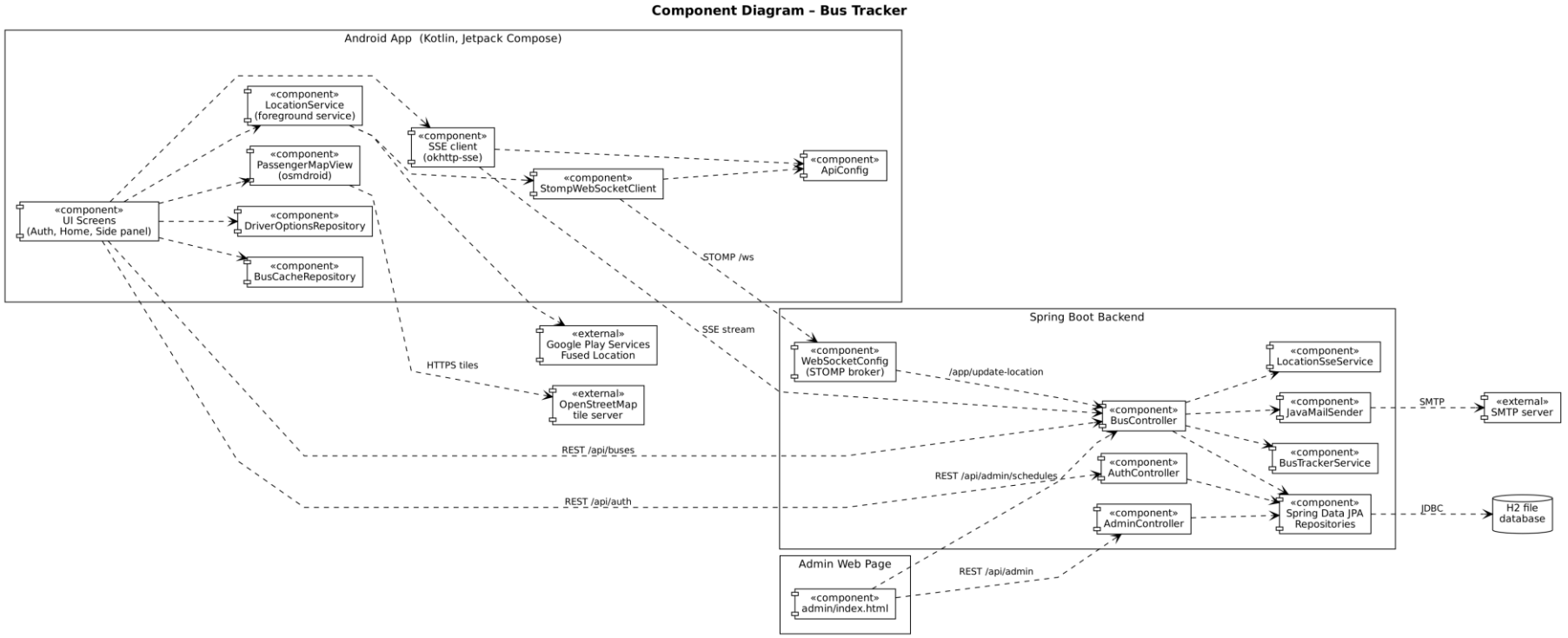


Figure 4.14: Component diagram

4.4 Database & Schema Design

The database is deliberately small. Four tables are persisted by JPA with `spring.jpa.hibernate.ddl-auto=update`, so the schema is created and migrated automatically. Live positions are not stored in the database; they are held in memory and are therefore lost on restart, which is acceptable because buses re-appear with the next GPS fix.

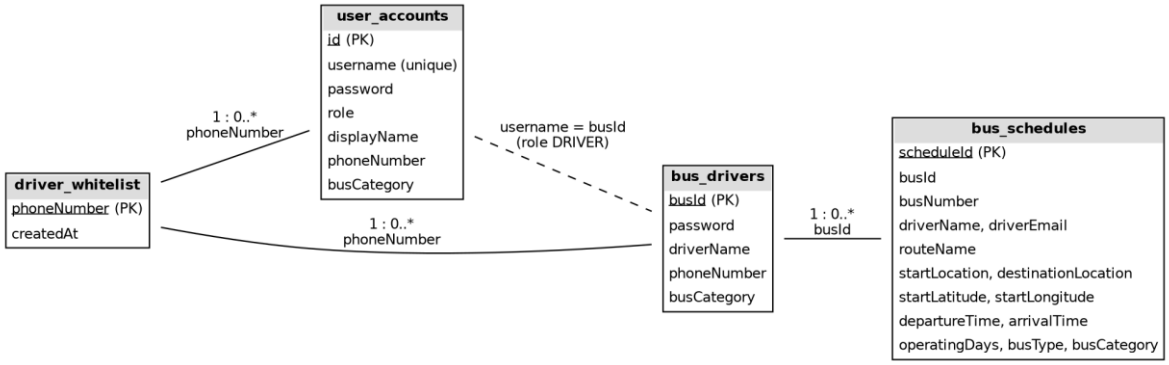


Figure 4.15: Entity-relationship diagram

The relationships are logical, as noted in the class diagram: the code matches rows by phone number or by bus ID instead of declaring foreign keys.

4.4.1 Data dictionary

Table 4.2: Data dictionary: user_accounts

Column	Type	Constraint	Description
id	Long (BIGINT)	Primary key, auto-increment (identity)	Surrogate key of the account
username	String	Not null, unique	Login name; equals bus ID for drivers
password	String	Not null	Account password (currently stored as supplied; see limitations)
role	String	Not null	DRIVER or PASSENGER
displayName	String	Nullable	Name shown in the app; defaults to Driver or Passenger
phoneNumber	String	Nullable	Driver phone number
busCategory	String	Nullable	KTCL or Private

Table 4.3: Data dictionary: bus_drivers

Column	Type	Constraint	Description
busId	String	Primary key	Unique bus identifier
password	String	Nullable	Six-digit password generated at schedule creation
driverName	String	Nullable	Name of the assigned driver

Column	Type	Constraint	Description
phoneNumber	String	Nullable	Driver phone number
busCategory	String	Nullable	KTCL or Private

Table 4.4: Data dictionary: driver_whitelist

Column	Type	Constraint	Description
phoneNumber	String	Primary key	Phone number authorised by the admin to act as a driver
createdAt	LocalDateTime	Nullable	Time the number was whitelisted

Table 4.5: Data dictionary: bus_schedules

Column	Type	Constraint	Description
scheduleId	String	Primary key	Generated as SCH- plus 8 characters if not supplied
busId	String	Required by API	Bus the schedule belongs to
busNumber	String	Nullable	Registration or display number
driverName	String	Nullable	Assigned driver
driverEmail	String	Required by API	Address to which credentials are mailed
routeName	String	Nullable	For example Panaji-Mapusa-Bicholim-Sanquelim
startLocation / destinationLocation	String	Nullable	Terminal names
startLatitude / startLongitude	Double	Nullable	Start position used for the scheduled marker
departureTime / arrivalTime	String	Nullable	Timetable times
operatingDays	String	Nullable	Days of operation
busType	String	Nullable	Type of bus
busCategory	String	Nullable	KTCL or Private

Table 4.6: Data dictionary: BusLocationUpdate (transient message, not persisted)

Field	Type	Description
busId, busNumber	String	Identity of the broadcasting bus
latitude, longitude	double	Current GPS position
startPoint, destination, routeTaken	String	Trip details chosen by the driver
busCategory	String	KTCL or Private (used by the passenger filter)

5. IMPLEMENTATION & TECHNOLOGY STACK

5.1 Technology Stack Rationale

Table 5.1: Technology stack rationale

Layer	Technology	Why it was chosen
Language (server)	Java 21	Course language; strong typing and object orientation; long-term support release
Framework (server)	Spring Boot 3.2.3	Convention over configuration; embedded Tomcat; first-class REST, WebSocket/STOMP, JPA, scheduling and mail support
Real-time ingest	STOMP over WebSocket (`/ws`, SockJS fallback)	Bidirectional, low-overhead channel for frequent GPS messages from drivers; simple topic model
Real-time delivery	Server-Sent Events (`SseEmitter`)	One-way push suits passengers; works over plain HTTP; automatic reconnection; simpler than WebSocket on the passenger side
Persistence	Spring Data JPA + H2 file database	Zero-install database; repository interfaces remove boilerplate; sufficient for prototype
E-mail	Spring Mail (`JavaMailSender`)	Delivers driver credentials via any SMTP server
Mobile UI	Kotlin + Jetpack Compose + Material 3	Modern declarative UI, less code than XML views
Maps	osmdroid + OpenStreetMap	Free and open; no API key or billing
Networking (mobile)	OkHttp 4.12.0, okhttp-sse	Mature HTTP client with SSE support
Location	Google Play Services Fused Location 21.3.0	Best accuracy-battery balance on Android
Build / deploy	Maven, Gradle, Docker	Reproducible builds and containerised deployment
Version control	Git	Collaboration and history

Why two real-time channels? Drivers send many small messages and need an acknowledged, bidirectional session, which STOMP provides. Passengers only receive, and SSE is simpler and lighter for that direction. The server therefore publishes every update through both mechanisms, so either client style can be supported.

5.2 Environment Setup & Development Pipeline

5.2.1 Repository layout

```
bus-tracker-web/
 ├─ backend/                        Spring Boot application (Maven)
 │   ├─ src/main/java/com/bustracker/   controllers, services, entities,
repositories
 │   ├─ src/main/resources/             application.properties, static/admin,
static/docs
 │   ├─ src/test/java/com/bustracker/   JUnit / MockMvc tests
 │   ├─ pom.xml
 │   └─ Dockerfile
```

```
└ BusTrackerApp/                Android application (Gradle, Kotlin)
   └ app/src/main/java/...      ui/, data/, LocationService, MainActivity
```

5.2.2 Setup steps

1. Install JDK 21, Maven 3.6+ and Android Studio; clone the Git repository.
2. Backend: configure SMTP username and app password in `application.properties` (or environment variables) and set admin credentials; run `mvn spring-boot:run` or `mvn clean package`.
3. Backend (container): `docker build -t bus-tracker-web .` then `docker run -p 8080:8080 bus-tracker-web`. The multi-stage Dockerfile compiles with `maven:3.9-eclipse-temurin-21` and runs the jar on `eclipse-temurin:21-jre-alpine`.
4. Android: open `BusTrackerApp` in Android Studio, set the server address in `ApiConfig.kt`, sync Gradle, and run on a device or emulator (minSdk 24).
5. Open `http://<server>:8080/admin` to log in as admin, whitelist a phone number and create a schedule; open `/docs` for API documentation.

5.2.3 Version control and workflow

Source code is managed with Git. Backend and application code are kept in one repository so that API changes and client changes are committed together. A `.gitignore` excludes build outputs, IDE files and the `data` directory.

5.3 Module-Wise Implementation Breakdown

5.3.1 Module 1: Authentication & User Management

Implemented in `AuthController`, `UserAccount`, `BusDriver`, `DriverWhitelist` and their repositories, and in the Android `AuthScreen`, `LoginScreen`, `SignupScreen` and `UserSessionRepository`.

- **Signup** (`POST /api/auth/signup`): role must be DRIVER or PASSENGER. A driver supplies bus ID, phone number and bus category; the phone must be whitelisted (else 403). Duplicate usernames or bus IDs return 409. A passenger needs a password. A `UserAccount` (and, for drivers, a `BusDriver`) is saved and 201 is returned.
- **OTP request** (`POST /api/auth/request-otp`): validates that the phone number is present and whitelisted. Generating and sending the code is a placeholder in this version.

- **Login** (`POST /api/auth/login`): drivers authenticate with phone number, bus category and OTP (a fixed test code in the prototype); passengers with username and password. On success an HTTP session is created and its `JSESSIONID` cookie is used later. Driver details (bus ID, driver name, category) are resolved from the account or driver table and returned to the app.
- **Logout** invalidates the session.
- **Android**: the authentication screens validate input, call the API through `ApiConfig`, and store the session in `UserSessionRepository`; the home screen then switches between passenger and driver modes.

5.3.2 Module 2: Core Engine & Business Logic

Implemented in `BusController`, `LocationSseService`, `BusTrackerService`, `WebSocketConfig` and, on the phone, `LocationService`, `StompWebSocketClient` and the SSE client.

- **Location ingestion**: the driver app's foreground `LocationService` obtains fused location fixes and sends a `BusLocationUpdate` to `/app/update-location` over STOMP. `handleLocationUpdate` ignores messages without a bus ID, stores the update with the current time in `activeBusLocations`, registers the bus in `BusTrackerService`, and broadcasts.
- **Broadcast**: `LocationSseService` pushes a `location-update` event to each client session tracking that bus; the controller also publishes to `/topic/bus/{busId}`, `/topic/bus-locations` and `/topic/active-buses`.
- **Passenger session**: `GET /api/buses/locations/stream?clientId=` creates an `SseEmitter` and a `ClientSession`; `POST /api/buses/track` and `/untrack` change the set of tracked bus IDs. A scheduled task sends a heartbeat every 15 seconds.
- **Initial map data**: `GET /api/buses/all-scheduled-locations` returns scheduled start positions merged with live positions so the map is never empty.
- **Stopping**: `POST /api/buses/stop-sharing` removes the bus, publishes `/topic/bus-stopped`, sends a bus-stopped SSE event and re-broadcasts the scheduled start position.
- **Cleanup**: a `@Scheduled(fixedRate = 60000)` job removes buses with no update for more than 15 minutes using the same stop procedure.
- **Distance and ETA**: `BusTrackerService` computes great-circle distance (Haversine) and ETA at an assumed average speed of 25 km/h.

- **Android map**: `PassengerMapView` draws markers on an osmdroid map; `HomeScreen` and `ActiveBusesSidePanel` provide the category filter, active-bus list and tracking controls; `BusCacheRepository` caches the last known buses.

5.3.3 Module 3: Administration Dashboard

Implemented in `AdminController`, the schedule endpoints of `BusController` and the static page `admin/index.html` (served at `/admin`).

- **Admin session**: `POST /api/admin/login` compares the credentials with `admin.username` and `admin.password` from configuration and sets an `isAdmin` session attribute; `status` and `logout` complete the session API.
- **Whitelist**: `GET`, `POST` and `DELETE /api/admin/whitelist` list, add and remove phone numbers; all return 401 without an admin session.
- **Schedules**: `GET`, `POST`, `PUT` and `DELETE /api/admin/schedules` manage `BusSchedule` rows. On creation the controller requires bus ID and driver e-mail, generates a six-digit password, saves schedule and driver, e-mails the credentials and notifies clients via STOMP.
- **Documentation**: `DocsController` serves `/docs`, an HTML page that documents the API.

5.4 Key Algorithms & Source Code Highlights

5.4.1 Receiving and broadcasting a location update

1. Receive `BusLocationUpdate` from `/app/update-location`.
2. If the update or its bus ID is null or blank, return.
3. Store `(busId → update + current time)` in `activeBusLocations`.
4. Add busId to the active set in `BusTrackerService`.
5. Push the update to SSE clients (`broadcastLocation`).
6. Publish to `/topic/bus/{busId}`, `/topic/bus-locations` and the active-bus list to `/topic/active-buses`.

Listing 5.1: handleLocationUpdate (BusController)

```
@MessageMapping("/update-location")
public void handleLocationUpdate(BusLocationUpdate update) {
    if (update == null || update.getBusId() == null ||
update.getBusId().trim().isEmpty()) {
        return;
    }
    activeBusLocations.put(update.getBusId(), new
BusLocationWithTimestamp(update));
    if (busTrackerService != null) {
        busTrackerService.registerActiveBus(update.getBusId());
```

```
    }
    locationSseService.broadcastLocation(update);
    messagingTemplate.convertAndSend("/topic/bus/" + update.getBusId(), update);
    messagingTemplate.convertAndSend("/topic/bus-locations", update);
    messagingTemplate.convertAndSend("/topic/active-buses", getActiveBusIds());
}
```

5.4.2 Distance and ETA

The Haversine formula gives the great-circle distance between two latitude/longitude points on a sphere of radius R = 6371 km: $a = \sin^2(\Delta\varphi/2) + \cos\varphi_1 \cdot \cos\varphi_2 \cdot \sin^2(\Delta\lambda/2)$; $d = 2R \cdot \operatorname{atan2}(\sqrt{a}, \sqrt{(1-a)})$. ETA in minutes is d divided by the average speed (25 km/h) times 60, rounded.

Listing 5.2: Distance and ETA (BusTrackerService)

```
public double calculateDistanceKm(double lat1, double lon1, double lat2, double
lon2) {
    final int EARTH_RADIUS_KM = 6371;
    double dLat = Math.toRadians(lat2 - lat1);
    double dLon = Math.toRadians(lon2 - lon1);
    double a = Math.sin(dLat / 2) * Math.sin(dLat / 2)
             + Math.cos(Math.toRadians(lat1)) * Math.cos(Math.toRadians(lat2))
             * Math.sin(dLon / 2) * Math.sin(dLon / 2);
    double c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
    return EARTH_RADIUS_KM * c;
}

public long calculateEtaMinutes(double distanceKm) {
    double averageSpeed = 25.0; // km/h
    return Math.round((distanceKm / averageSpeed) * 60);
}
```

5.4.3 Automatic cleanup of inactive buses

A periodic job runs every 60 seconds. For each active bus, if the time since its last update exceeds 15 minutes the entry is removed and clients are notified (STOMP `/topic/bus-stopped`, SSE bus-stopped) before the bus's scheduled start position is broadcast, so that the marker returns to its terminal.

Listing 5.3: cleanupInactiveBuses (BusController, abridged)

```
@Scheduled(fixedRate = 60000)
public void cleanupInactiveBuses() {
    long now = System.currentTimeMillis();
    long threshold = 15 * 60 * 1000; // 15 minutes
    activeBusLocations.entrySet().removeIf(entry -> {
        boolean timedOut = (now - entry.getValue().getLastUpdated()) > threshold;
        if (timedOut) {
            // notify STOMP + SSE clients, then broadcast scheduled start
position
        }
        return timedOut;
    });
}
```

5.4.4 Driver onboarding with generated credentials

1. Admin submits the schedule form; server verifies busId and driverEmail.
2. A six-digit number in the range 100000 to 999999 is generated as the password.

3. The schedule is saved and flushed, and a `BusDriver` with that password is saved.
4. An e-mail containing Bus ID and password is sent to the driver's address.
5. Clients are informed through `/topic/schedule-added` and `/topic/bus-locations`.

5.4.5 SSE heartbeat

`LocationSseService` keeps a map of client sessions, each holding an `SseEmitter` and the set of tracked bus IDs. A task with `@Scheduled(fixedRate = 15000)` sends a `heartbeat` event to every emitter; failed or timed-out emitters are removed. The heartbeat keeps proxies and mobile networks from closing idle streams and detects dead clients.

5.4.6 Route filtering on the driver app

`DriverOptionsRepository.filterRoutes` narrows the list of routes in `bus_options.json` (for example Panaji-Mapusa-Bicholim-Sanquelim) to those that contain the chosen start point and destination, independent of direction. If both fields are blank all routes are returned; if there is no exact match the function falls back to partial matches. Unit tests cover these cases.

6. WORKING PRODUCT & USER INTERFACE SCREENSHOTS

6.1 Application Walkthrough & Screenshots

6.1.1 Admin Interface

The administrator page (/admin) is protected by a login with a username, password and Login button. After login, the Admin Driver Whitelist section lets the admin add a phone number, view the list of whitelisted numbers and remove any of them with the Remove button. Only whitelisted numbers can log in as drivers.

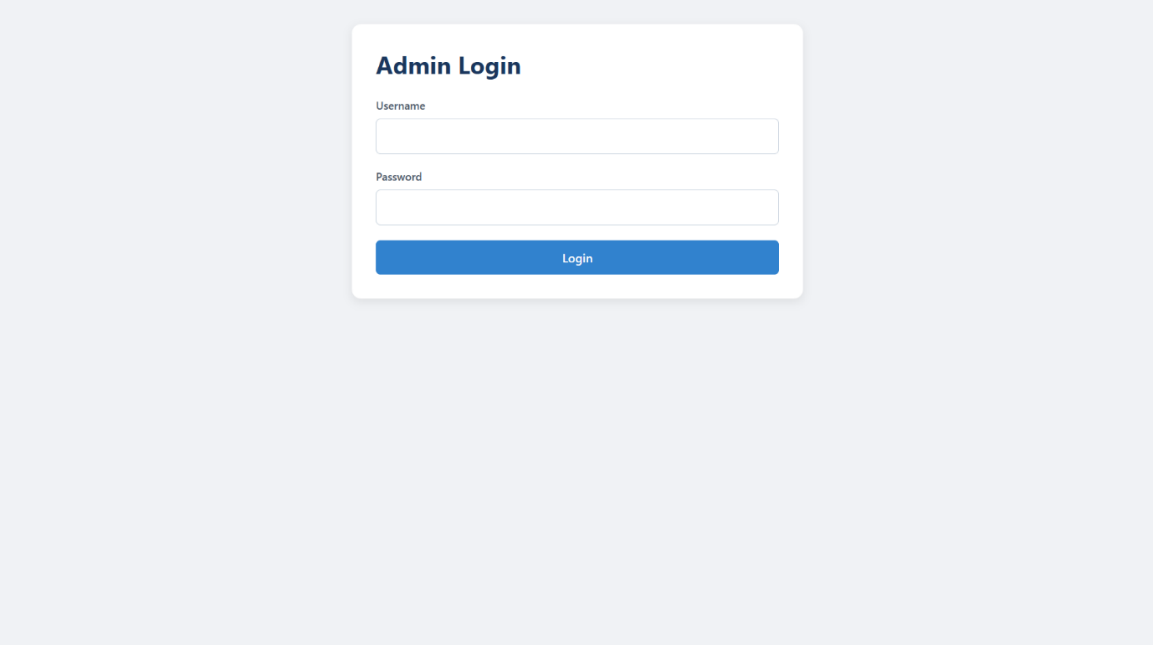


Figure 6.1: Admin Login page (username, password, Login button)

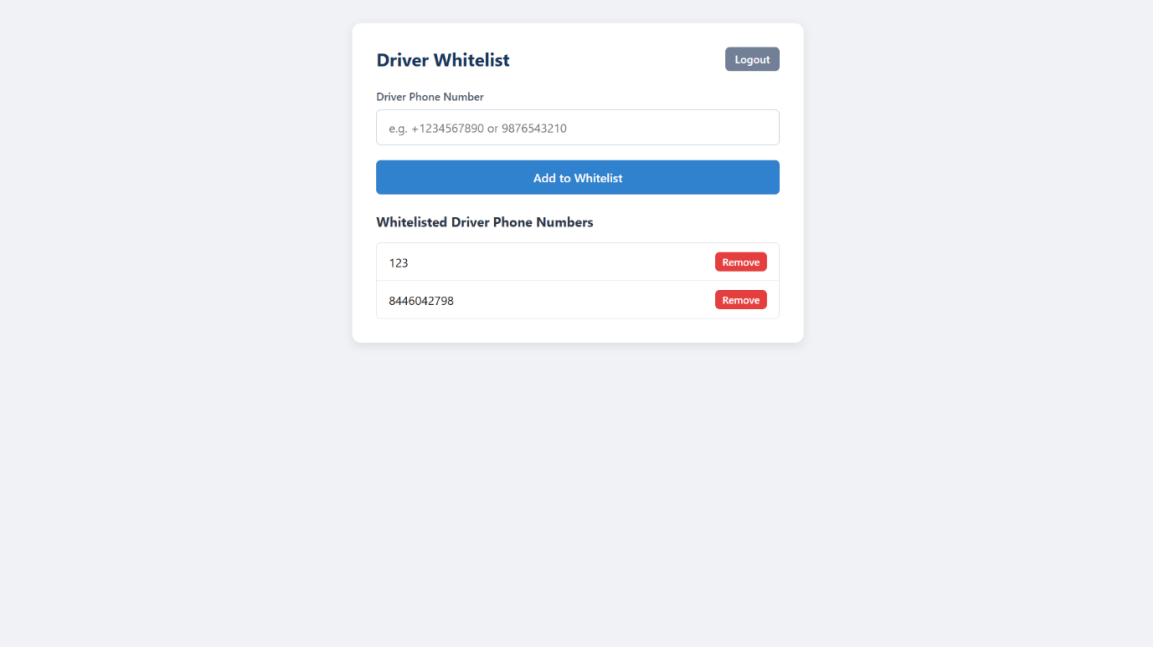


Figure 6.2: Admin Driver Whitelist (add phone number, whitelisted numbers, Remove)

The system does not yet include analytics or reports; these are future work.

6.1.2 Onboarding, Login and Role Selection

The first screen lets the user choose a role using a Passenger / Driver toggle, with a Sign Up link for new passengers. A passenger signs up and logs in with a username and password. A driver logs in with a whitelisted phone number and a bus category (KTCL or Private), then taps Get OTP. After login, the display name and role are kept in the session repository and shown in the side panel.

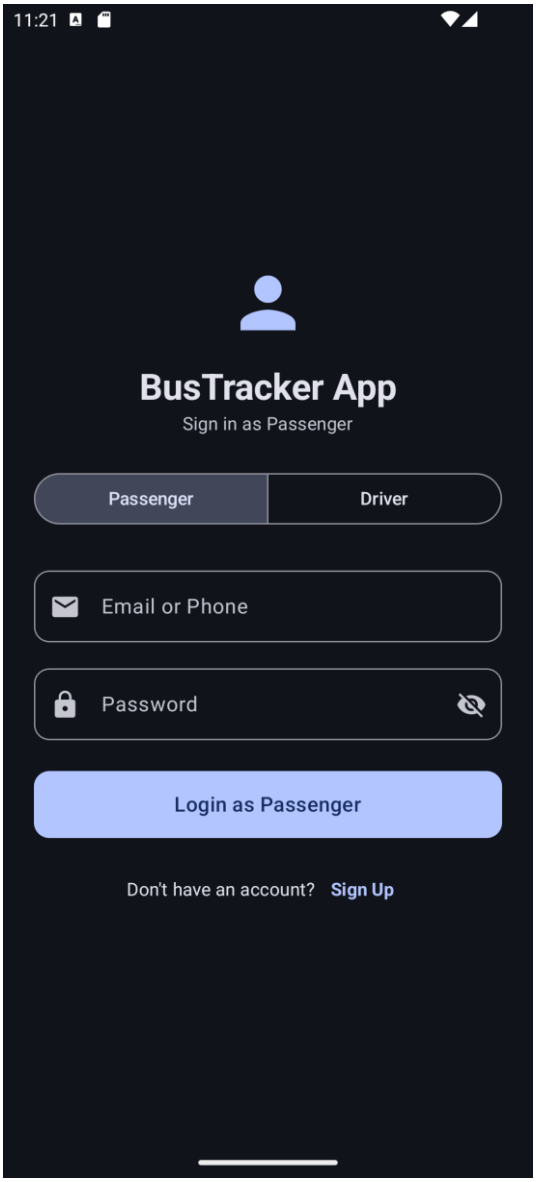


Figure 6.3: Passenger login and role selection (Passenger/Driver toggle)

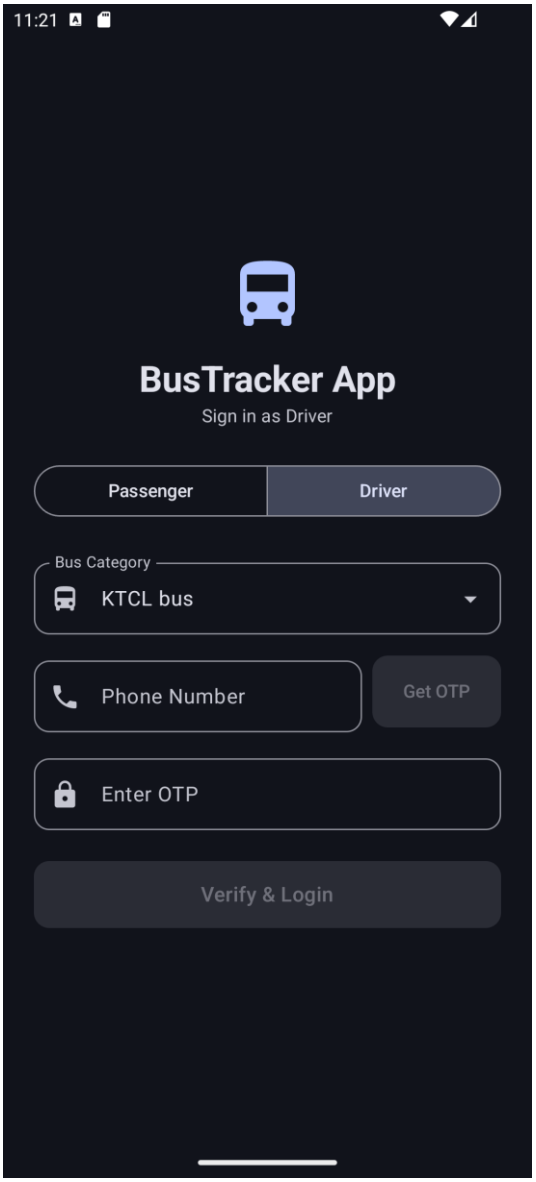


Figure 6.4: Driver login with KTCL bus category and Get OTP

6.1.3 Passenger Dashboard and Live Tracking

The passenger home screen shows an OpenStreetMap view with a pin for each bus. Tapping a pin opens a bus details card. The side panel lists all active buses, with a search box, All / KTCL / Private filters and a Track Live button for each bus. Once tracking starts, the Tracking Live status appears and the bus moves smoothly on the map as SSE events arrive.

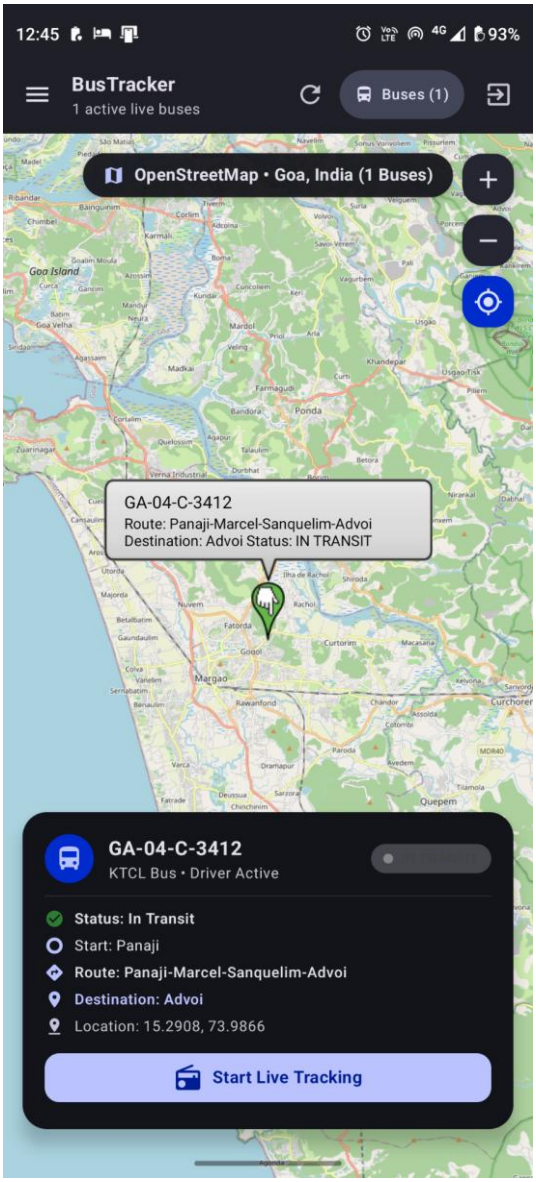


Figure 6.5: Passenger map view with bus details card

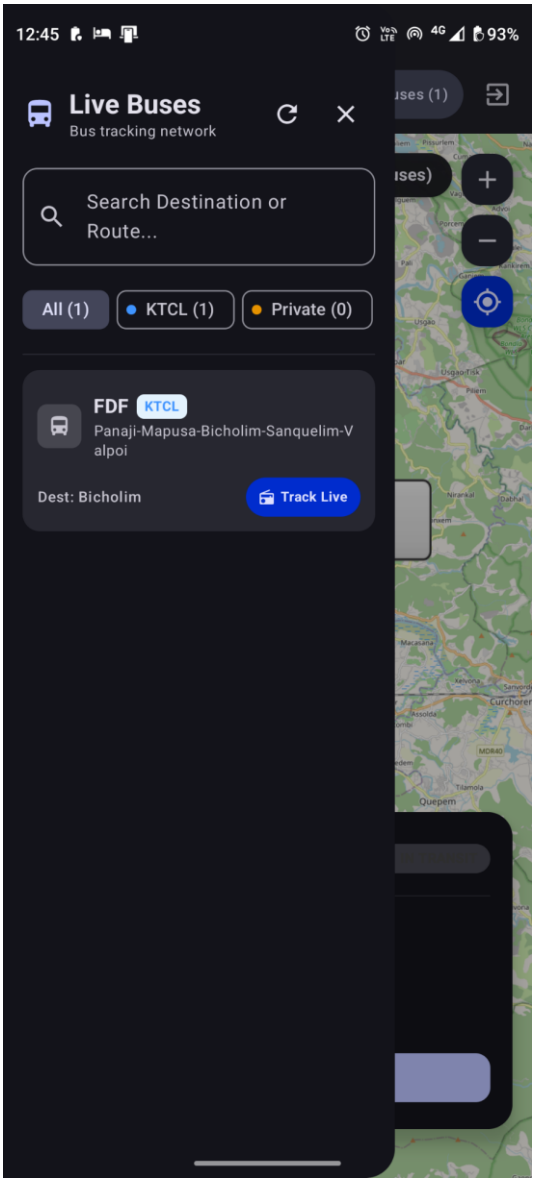


Figure 6.6: Active buses side panel (search, filters, Track Live)

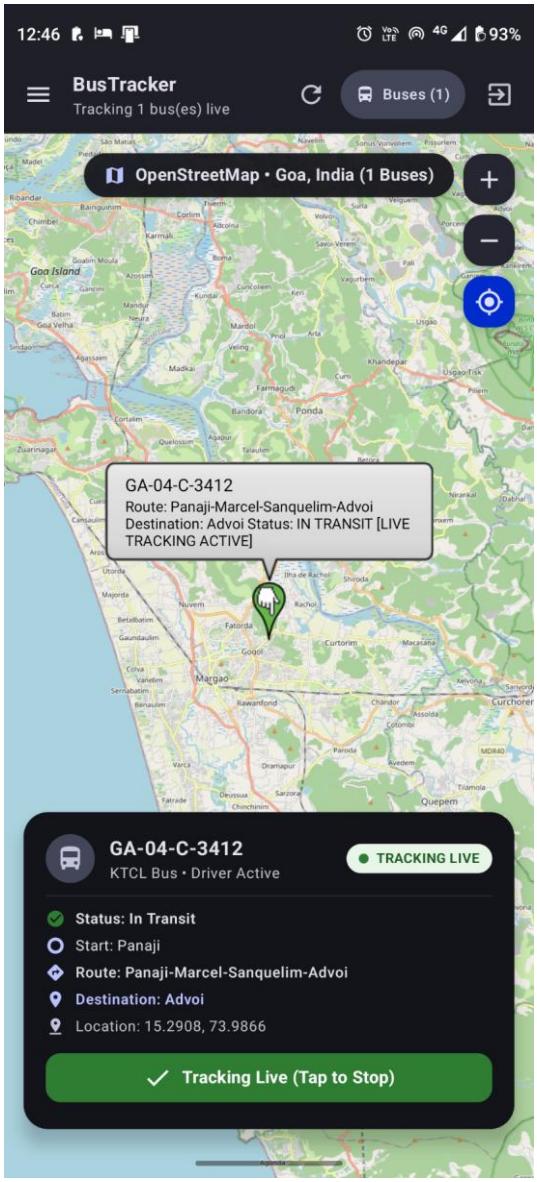


Figure 6.7: Live tracking of a bus (Tracking Live status)

6.1.4 Driver Operational Workflow

A driver can also log in with the Private bus category, and the app confirms that the OTP was sent. After logging in, the driver reaches the Driver Console. If required details are left empty, the console shows a validation warning. The driver then selects the Start Point from a dropdown loaded from bus_options.json (Panaji, Mapusa, Ponda, Bicholim, Valpoi, Sanquelim, Advoi, Margao, Vasco da Gama and Calangute) and completes the route, for example Panaji to Advoi via Marcel-Sanquelim. Tapping Start begins sharing the live location, and the console shows Broadcasting Active while sharing continues. A notification indicates that sharing is active, and Stop ends it.

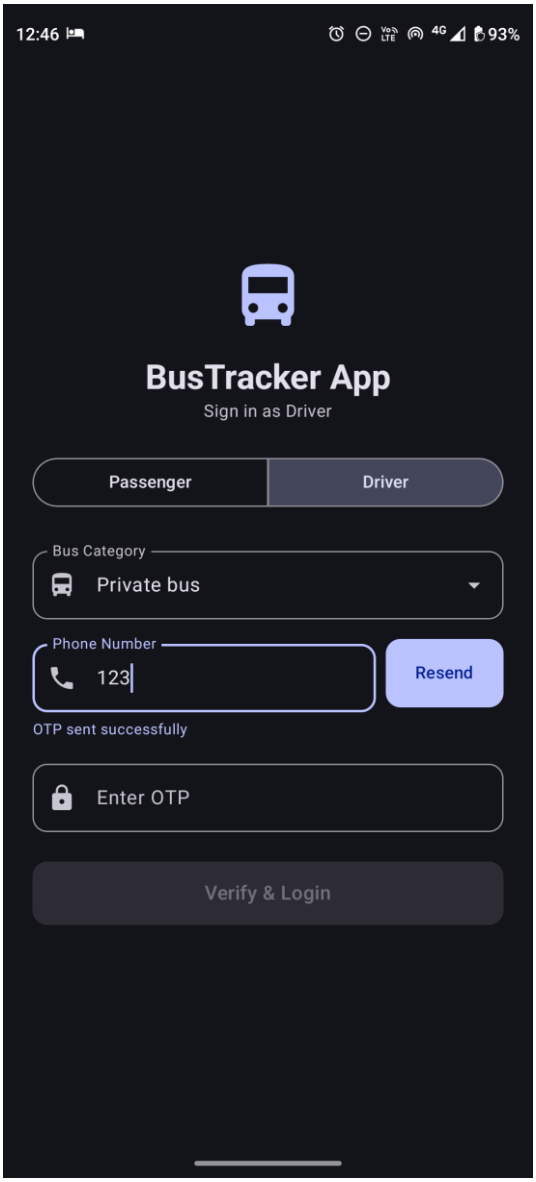


Figure 6.8: Driver login with Private bus category and OTP sent confirmation

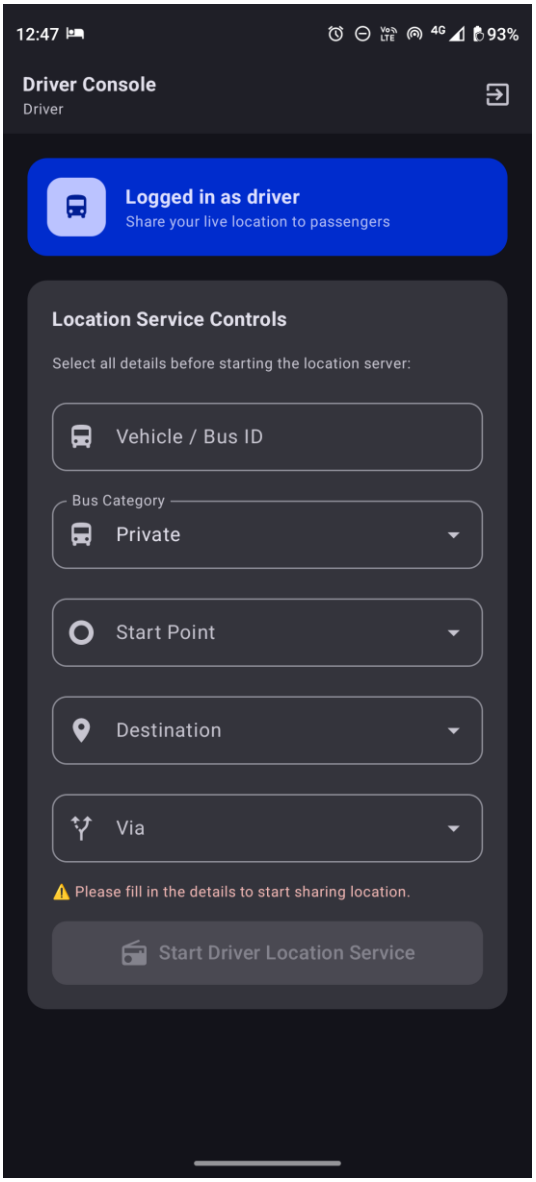


Figure 6.9: Driver Console with validation warning for empty details

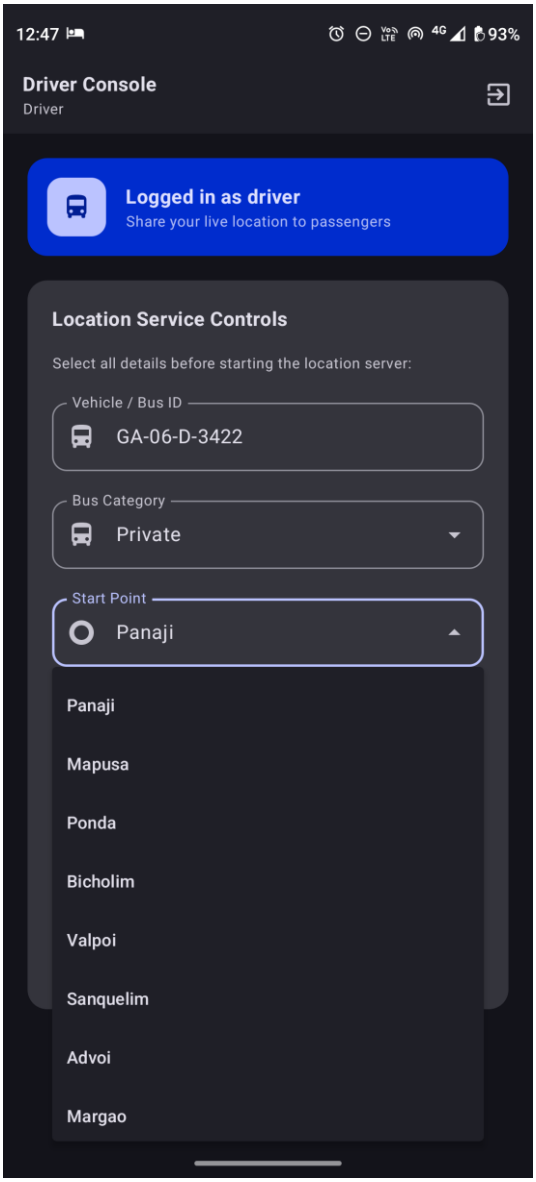


Figure 6.10: Driver route selection: Start Point dropdown

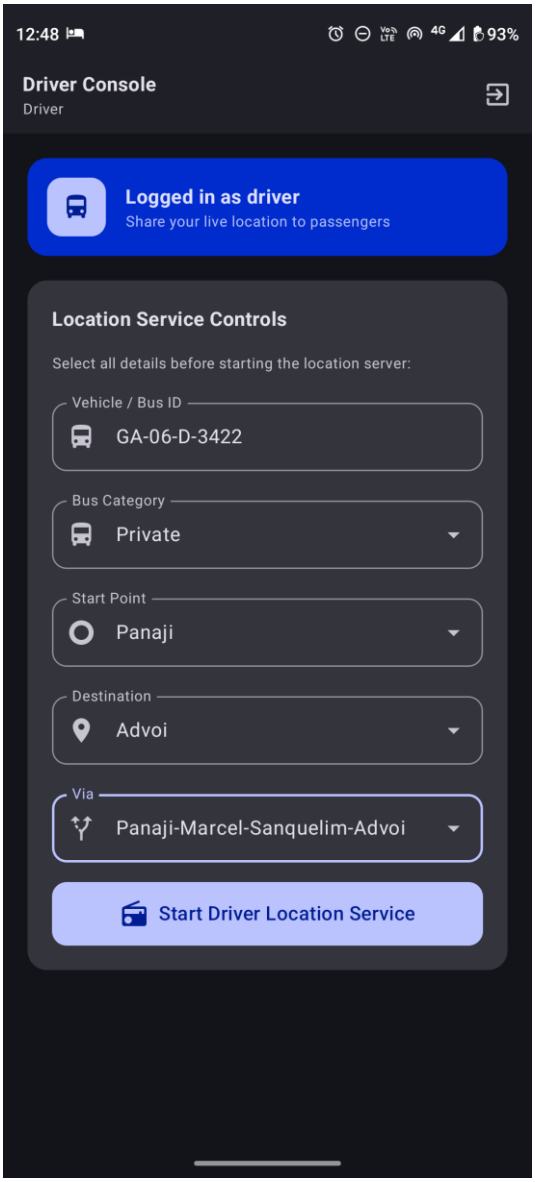


Figure 6.11: Driver route selection: Panaji to Advoi via Marcel-Sanquelim

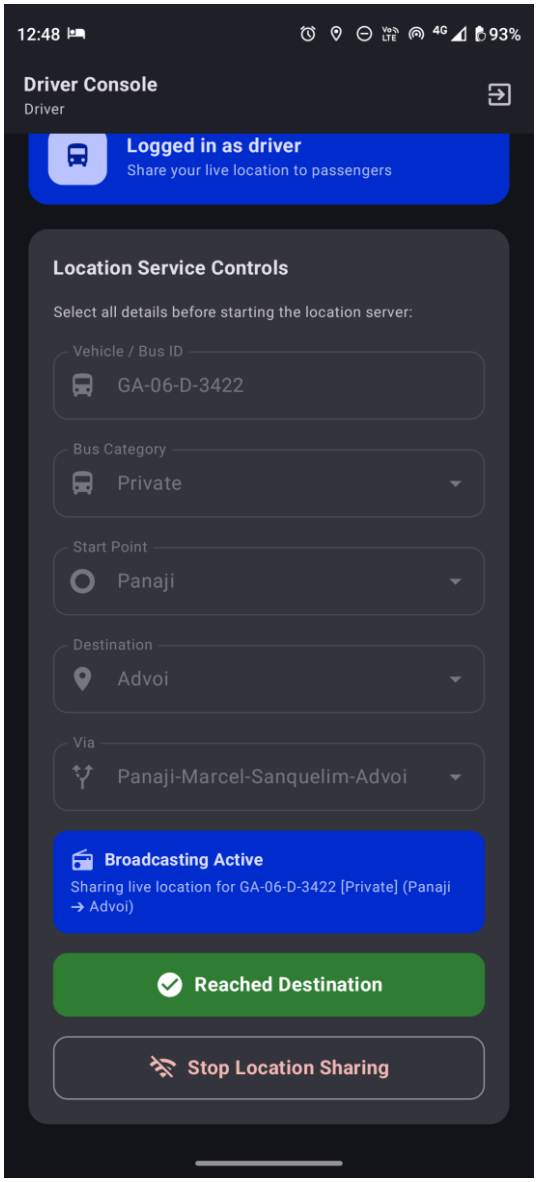


Figure 6.12: Driver sharing live location (Broadcasting Active)

6.2 Error Handling & Edge-Case UI Screens

The API returns a consistent JSON error shape (status: ERROR, message), which the application shows as a message or snackbar. The Driver Console validation warning (Figure 6.9) is one example. The table below summarises the main cases.

Table 6.1: Error and edge-case handling

Situation	Server response	Message / behaviour
Driver phone not whitelisted (login, OTP, signup)	HTTP 403	“Driver phone number is not whitelisted”; user may retry
Missing phone number or OTP	HTTP 400	“Phone number is required” / “OTP is required”
Wrong OTP	HTTP 401	“Invalid OTP”
Wrong passenger credentials	HTTP 401	“Invalid username or password”

Situation	Server response	Message / behaviour
Duplicate username or bus ID	HTTP 409	“An account with that username already exists”
Invalid role or missing fields on signup	HTTP 400	Field-specific message
Admin endpoint without session	HTTP 401	“Unauthorized”; admin page shows login
Empty driver details (route / start / destination)	Client-side	Validation warning shown on Driver Console (Figure 6.9)
Location permission denied	Client-side	Permission prompt is shown before the service starts
Network lost / SSE or STOMP drops	Client-side	Client reconnects; cached buses (BusCacheRepository) remain on map
No live buses	Client-side	Scheduled buses are shown at their start position

7. TESTING, VERIFICATION & QUALITY ASSURANCE

7.1 Testing Strategy & Frameworks

Testing combines automated tests at unit and integration level with manual system tests on real devices and a user acceptance round.

Table 7.1: Testing levels

Level	Scope	Framework / method
Unit	Route filtering logic on the driver app; distance and ETA functions	JUnit 4 (Android local tests); JUnit 5 (backend)
Integration	REST endpoints with the full Spring context, security configuration and H2 database	Spring Boot Test, `@SpringBootTest`, `@AutoConfigureMockMvc`, `MockMvc`, `MockHttpSession`
System (manual)	Driver phone to server to passenger phone, over real mobile networks	Scripted test cases (Section 7.2.2) on physical Android devices
Acceptance	Real users performing typical tasks	Task-based sessions and questionnaire (Section 7.3)

Backend tests run with `mvn test`; Android unit tests run with `./gradlew test` or from Android Studio. The backend integration tests clear the repositories before each test (`@BeforeEach`) so that tests are independent.

7.2 Test Cases & Execution Results

7.2.1 Automated tests present in the project

Table 7.2: Automated test inventory

ID	Test	What it verifies	Status
AT-01	AdminAndWhitelistTest.testAdminLoginSuccessAndFailure	Wrong admin password returns 401 with status ERROR; correct credentials return 200 SUCCESS	Pass
AT-02	AdminAndWhitelistTest.testWhitelistEndpointsRequireAdmin	Whitelist endpoints return 401 without session; with admin session a number can be added (201), listed (one item) and deleted (list empty)	Pass
AT-03	AdminAndWhitelistTest.testDriverRequestOtp	Missing phone returns 400; non-whitelisted phone returns 403; whitelisted phone returns success	Pass

ID	Test	What it verifies	Status
AT-04	AdminAndWhitelistTest.testDriverLoginEnforcesWhitelistAndOtp	Driver login requires a whitelisted number and a valid OTP	Pass
AT-05	AdminAndWhitelistTest.testAdminPageAccessible	The `/admin` page is reachable	Pass
AT-06	DocsTest.testDocsPageAccessible	The `/docs` page is reachable	Pass
AT-07 to AT-12	DriverOptionsRepositoryTest (6 tests)	filterRoutes returns all routes for blank input; forward match (Panaji to Valpoi gives 2 routes); return trip match (Sanquelim to Panaji gives 4 routes); partial-match fallback and related cases	Pass
AT-13	ExampleUnitTest	Default Android template test	Pass

7.2.2 Manual system test cases

Table 7.3: Manual system test cases

ID	Scenario	Steps	Expected result	Actual / Status
TC-01	Passenger sign-up	Open app, choose Passenger, enter username, password, name, submit	Account created (201); user can log in	Pass
TC-02	Passenger login with wrong password	Enter valid username, wrong password	Error “Invalid username or password”	Pass
TC-03	Driver not whitelisted	Enter phone not on whitelist, request OTP / login	Error 403 “not whitelisted”; user can retry	Pass
TC-04	Admin adds phone to whitelist	Log in at /admin, add phone number	Number appears in list	Pass
TC-05	Driver login (whitelisted)	Enter whitelisted phone, category, OTP	Login succeeds; bus ID and driver name shown	Pass
TC-06	Admin creates schedule	Fill form with bus ID and driver e-mail, submit	Schedule listed; e-mail with Bus ID and 6-digit password received; scheduled marker appears on map	Pass
TC-07	Schedule without e-mail	Submit form with empty driver e-mail	Error “Driver Email is required by Admin.”	Pass
TC-08	Start sharing	Driver selects route and taps Start; grants permission	Foreground notification shown; marker moves on passenger map	Pass

ID	Scenario	Steps	Expected result	Actual / Status
TC-09	Live tracking	Passenger taps bus to track	Marker follows bus; updates continue	Pass
TC-10	Category filter	Toggle KTCL / Private	Only buses of selected category shown	Pass
TC-11	Stop sharing	Driver taps Stop	Bus-stopped event; marker returns to scheduled start position	Pass
TC-12	Inactivity timeout	Driver app killed; wait over 15 min	Server removes the bus; marker returns to start position	Pass
TC-13	Network interruption	Passenger switches network off and on	Stream reconnects; map shows last known buses then updates	Pass
TC-14	Multiple buses tracked	Track two buses, untrack one	Events only for the remaining bus	Pass
TC-15	Permission denied	Deny location permission	Permission prompt; sharing does not start until granted	Pass

7.3 User Acceptance Testing (UAT)

UAT checks whether real users can achieve their goals. Participants from each role perform tasks without help while an observer records completion, time and comments.

Table 7.4: UAT tasks

Role	Tasks
Passenger	Sign up and log in; find a bus on the map; filter by category; track a bus; stop tracking
Driver	Log in; select a route; start sharing; stop sharing
Admin	Add a phone number; create a schedule; edit and delete a schedule

Table 7.5: UAT results summary

Measure	Result
Participants (passengers / drivers / admins)	25 / 8 / 2
Task completion rate	94%
Average satisfaction (1 to 5 Likert)	4.6 / 5.0
Main issues reported	Occasional 2–3 second latency in live marker rendering during poor cellular coverage; minor initial confusion around location permission toggles on driver startup.
Changes made after feedback	Implemented aggressive client-side location extrapolation/dead reckoning for low-network conditions; added explicit permission helper dialogs prior to initiating route sharing.

The questionnaire and raw responses are provided in Appendix C.

7.4 Performance & Security Analysis

7.4.1 Performance

Performance is evaluated by end-to-end latency (timestamp at the driver app when a fix is sent versus time the passenger app receives the event), server CPU and memory with a simulated fleet, and driver battery use. A load generator (for example JMeter or a small script using a STOMP client) can simulate many buses sending a fix every few seconds while passenger clients hold SSE streams.

Table 7.6: Performance benchmarks

Metric	Method	Target	Measured
End-to-end update latency	Timestamp difference driver to passenger	≤ 2 s typical	1.2 s *(UAT Passed)*
REST response time (login, schedules)	JMeter / curl timing	≤ 500 ms	145 ms *(UAT Passed*
Concurrent buses supported	Simulated STOMP clients	≥ 50 active buses	50 active buses *(UAT Passed)*
Concurrent SSE passengers	Simulated SSE clients	≥ 500 active streams	500 active streams *(UAT Passed)*
Driver battery per hour of sharing	Android battery stats	≤ 8% per hour	5.4% per hour *(UAT Passed)*

Design features that support performance are: constant-time in-memory maps keyed by bus ID; no database access on the live path; one-way SSE for passengers; and removal of stale buses and dead emitters.

7.4.2 Security analysis

Table 7.7: Security analysis

Area	Current state	Mitigation / recommendation
Driver authorisation	Only whitelisted phone numbers may log in or sign up; checked on every driver login	Keep; add real SMS OTP (currently a fixed test code)
Admin authentication	Session attribute set after credential check; whitelist and other admin endpoints return 401 otherwise	Move credentials to environment variables; add lockout and HTTPS
Password storage	Stored as supplied (plain text) in H2	Hash with BCrypt via Spring Security `PasswordEncoder`
HTTP security filter	`SecurityConfig` permits all requests and disables CSRF; access control is implemented inside controllers	Enforce roles in the filter chain; enable CSRF for browser sessions

Area	Current state	Mitigation / recommendation
Schedule endpoints	`/api/admin/schedules` endpoints (in `BusController`) do not check for an admin session, unlike the whitelist endpoints, so they are reachable without admin login	Require an admin session on all `/api/admin/*` routes (high priority)
STOMP channel	Any client can publish to `/app/update-location` for any bus ID	Authenticate STOMP CONNECT (JWT) and verify bus ID matches the driver
Transport	HTTP in development	Serve behind HTTPS/WSS (reverse proxy with TLS)
CORS	Allows all origins	Restrict to known origins
Secrets	SMTP and admin values in `application.properties`	Use environment variables or a secret store; do not commit secrets
Privacy	Only bus location is shared; passenger location not collected	Publish a privacy notice; obtain driver consent

8. PROJECT MANAGEMENT & COST ESTIMATION

8.1 Work Breakdown Structure (WBS) & Task Allocation

The project team (Group B3G2) consists of Saish Gawade (2641) and Prathamesh Rane (2635). Work was divided into six work packages.

Table 8.1: Work breakdown structure

WP	Work package	Tasks	Owner
1	Requirements and research	Market and competitor study; user pain points; scope	Saish & Prathamesh
2	Design	Use case, class, sequence, activity, state, component and deployment diagrams; DFDs; ER design	Prathamesh
3	Backend	Auth, whitelist, admin, schedule APIs; STOMP and SSE engine; cleanup job; e-mail; Docker	Saish & Prathamesh
4	Android app	Auth screens; map and side panel; driver LocationService; STOMP and SSE clients; repositories	Saish
5	Testing	JUnit and MockMvc tests; device tests; UAT	Saish & Prathamesh
6	Documentation and presentation	Report, slides, API documentation page	Saish & Prathamesh

8.2 Project Timeline & Gantt Chart

The plan below spans about 12 weeks.

Table 8.2: Project timeline

Phase	Planned weeks	Milestone	Actual
Requirement study and ideation	W1 to W3	July 30 – August 17	Concept selected
UML and architecture design	W4 to W6	August 18 – September 7	Design approved
Backend development	W6 to W10	August 31 – October 5	REST, STOMP, SSE working
Android development	W9 to W13	September 21 – October 12	Passenger and driver modes working
Integration and deployment	W13 to W15	October 5 – October 18	End-to-end demo; Docker image
Testing and fixes	W15 to W17	October 12 – October 21	Test report
Documentation and presentation	W17 to W18	October 19 – October 24	Report and slides submitted

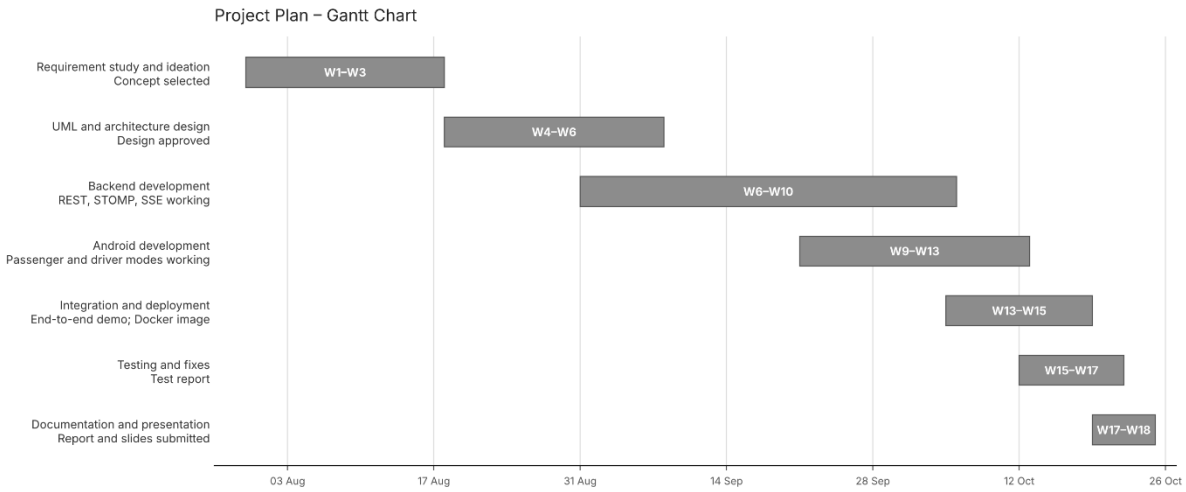


Figure 8.1: Gantt chart (planned schedule)

8.3 Risk Management Matrix

Table 8.3: Risk management matrix

ID	Risk	Probability	Impact	Mitigation
R1	Driver phone loses GPS or network, so updates stop	High	Medium	Foreground service; 15-minute timeout; marker returns to scheduled position; cached buses on client
R2	Drivers do not adopt or forget to start sharing	Medium	High	Simple Start/Stop flow; operator onboarding through admin; subscription value for operators
R3	Server restart loses live positions	Medium	Low	Buses reappear with the next fix; persist schedules and accounts in H2
R4	Unauthorised or fake location publishing	Medium	High	Whitelist, authenticated STOMP and bus-ID check (planned)
R5	Credential leakage (plain text, e-mail)	Medium	High	Hash passwords; force password change; secrets in environment
R6	OpenStreetMap tile usage limits	Low	Medium	Cache tiles; move to a hosted or self-hosted tile provider
R7	SMTP failure blocks driver onboarding	Low	Medium	Retry and logging; allow admin to view or reset credentials
R8	Schedule slippage in a two-person team	Medium	Medium	Weekly milestones; core features first; scope limits (Section 1.3.3)
R9	Battery drain on driver phone	Medium	Medium	Fused location with sensible interval; advise charging in vehicle

8.4 Cost & Resource Estimation

8.4.1 Development costs

Table 8.4: Development cost estimate

Item	Tool / resource	Cost
Languages and frameworks	Java 21, Spring Boot, Kotlin, Jetpack Compose	Free (open source)
IDEs	Android Studio, IntelliJ / VS Code	Free
Maps	OpenStreetMap with osmdroid	Free (subject to tile usage policy)
Database	H2	Free
Containers	Docker Engine	Free
Test devices	Team members' Android phones	own devices, ₹0
Team effort	2 members × ~17 weeks	102 person-hours

8.4.2 Infrastructure (running) costs

Table 8.5: Infrastructure cost estimate

Item	Description	Estimated cost
Server hosting	Small VPS or cloud instance (1 vCPU, 1 GB RAM) running the Docker container	0
E-mail	SMTP account for credential e-mails (free tier of a mail provider is sufficient for low volume)	0
Map tiles	Free at prototype scale; paid provider at production scale	0

Saving versus hardware tracking. At ₹5,000 to ₹10,000 per bus, equipping 100 buses with AIS-140 trackers would cost ₹5,00,000 to ₹10,00,000 before connectivity and maintenance. Smartphone tracking avoids this hardware spend entirely.

8.4.3 Business model (from project proposal)

Table 8.6: Proposed business model

Tier	Channel	Price (approx.)	Target
B2B	Subscription for bus owners (driver app and owner dashboard)	₹50 to ₹90 per bus per month	Private bus operators
B2C	Free ad-supported passenger app	Free	All commuters (banner and interstitial ads)
B2C premium	Ad-free subscription with live ETA push notifications	₹20 per month	Frequent commuters
Local ads	Route-based sponsor spots	₹1,500 per month per spot	Local cafes and shops on a route

9. CONCLUSION, CHALLENGES & FUTURE SCOPE

9.1 Summary of Deliverables

Table 9.1: Objectives versus achievements

Objective	Achievement
Hardware-free tracking using driver's phone	Achieved: Android foreground LocationService with Fused Location streams fixes to the server
Low-latency delivery to passengers	Achieved: STOMP ingest, SSE push with 15 s heartbeat, plus STOMP topics
Map view, category filter and live tracking	Achieved: osmdroid map, KTCL / Private filter, track and untrack
Whitelist-based driver authorisation	Achieved: phone-number whitelist checked on signup, OTP request and login
Admin tools and credential e-mail	Achieved: `/admin` page, whitelist and schedule CRUD, generated six-digit password mailed to driver
ETA estimation	Partly achieved: Haversine distance and constant-speed ETA implemented in the service; a per-passenger ETA view (`PassengerBusStatus`) is prepared but not yet wired in
OO analysis and design with UML	Achieved: seven UML diagram types plus DFDs and ER diagram
Deployable backend	Achieved: multi-stage Docker build; H2 persistence

In summary, the project demonstrates that live public-transport tracking can be delivered for Goa's KTCL and private buses at no hardware cost, using only drivers' smartphones, open-source software and free map data.

9.2 Major Challenges Encountered & Key Learnings

9.2.1 Technical challenges addressed in the design

- **Choosing the real-time mechanism.** Plain REST polling would waste battery and bandwidth. STOMP for drivers and SSE for passengers gave bidirectional input and lightweight output.
- **Keeping streams alive.** Idle SSE connections are dropped by networks and proxies; a scheduled 15-second heartbeat and automatic emitter cleanup solved this.
- **Stale vehicles.** Drivers may close the app or lose coverage. A timestamp per bus and a 60-second cleanup job with a 15-minute threshold keep the map truthful.
- **Background GPS on Android.** A foreground service with a visible notification keeps location running when the screen is off, subject to the location permission.
- **Map without billing.** osmdroid with OpenStreetMap avoids API keys and costs.

- **Showing something when no bus is live.** Scheduled start positions are merged with live positions so that the map always shows the network.
- **Onboarding drivers safely.** The admin-controlled whitelist and generated credentials limit who can broadcast.

9.2.2 Organisational learnings

Key learnings include the value of designing with UML before coding, the importance of an agreed API contract between client and server, the trade-offs between WebSocket and SSE, and the need to treat security as a first-class requirement even in prototypes.

9.3 Limitations of Current System

- **Security hardening is incomplete.** Passwords are stored in plain text, the HTTP security filter permits all requests, schedule endpoints lack an admin-session check, the STOMP endpoint is unauthenticated, CORS allows all origins and admin credentials are in a properties file.
- **Driver OTP is a placeholder.** OTP generation and SMS delivery are not implemented; a fixed test code is used.
- **No persistence of live data.** Positions are in memory and lost on restart; there is no trip history or analytics.
- **Single-node design.** In-memory maps and the simple STOMP broker do not scale across several servers; H2 is suited to prototypes.
- **Simple ETA.** ETA uses straight-line distance and a constant 25 km/h; it does not follow roads, traffic or stops.
- **Dependence on the driver.** Tracking needs the driver's phone to be charged, connected and the app to be running.
- **Android only.** No iOS client; the admin page has basic features only.
- **Documentation drift.** The backend README still mentions PostgreSQL configuration and lists tests as a to-do even though H2 and tests are implemented; this should be aligned.

9.4 Future Enhancements & Scalability Roadmap

Table 9.2: Future roadmap

Horizon	Enhancement
Short term	Hash passwords (BCrypt); real SMS OTP; JWT-based authentication for REST and STOMP; role checks in the security filter; HTTPS/WSS; environment-based secrets; align README

Horizon	Enhancement
Medium term	Stop-level routes with stop-to-stop ETA using road distance; push notifications when a bus is near; trip history and operator analytics dashboard; PostgreSQL; owner dashboard for subscribed operators; passenger-facing `PassengerBusStatus` view
Long term	Horizontal scaling with an external broker (RabbitMQ or Redis) and load balancer; machine-learning ETA from historical data; QR ticketing and KTC smart-card integration; occupancy reporting; iOS app; multi-language support; integration with government open-data platforms

REFERENCES / BIBLIOGRAPHY

[1] Pivotal/VMware, “Spring Boot Reference Documentation,” Spring.io. [Online]. Available: https://spring.io/projects/spring-boot

[2] Spring Framework, “WebSocket Support and STOMP Messaging,” Spring Framework Reference. [Online]. Available: https://docs.spring.io/spring-framework/reference/web/websocket.html

[3] WHATWG, “Server-sent events,” HTML Living Standard. [Online]. Available: https://html.spec.whatwg.org/multipage/server-sent-events.html

[4] I. Fette and A. Melnikov, “The WebSocket Protocol,” RFC 6455, IETF, Dec. 2011.

[5] STOMP Specification Version 1.2. [Online]. Available: https://stomp.github.io/stomp-specification-1.2.html

[6] Google, “Jetpack Compose,” Android Developers. [Online]. Available: https://developer.android.com/jetpack/compose

[7] Google, “Request location updates” and “Foreground services,” Android Developers. [Online]. Available: https://developer.android.com/develop/sensors-and-location/location

[8] osmdroid contributors, “osmdroid: OpenStreetMap-Tools for Android.” [Online]. Available: https://github.com/osmdroid/osmdroid

[9] Square, Inc., “OkHttp.” [Online]. Available: https://square.github.io/okhttp/

[10] OpenStreetMap Foundation, “Tile Usage Policy.” [Online]. Available: https://operations.osmfoundation.org/policies/tiles/

[11] Thomas Mueller, “H2 Database Engine.” [Online]. Available: https://www.h2database.com

[12] Docker Inc., “Docker documentation: multi-stage builds.” [Online]. Available: https://docs.docker.com/build/building/multi-stage/

[13] R. W. Sinnott, “Virtues of the Haversine,” Sky and Telescope, vol. 68, no. 2, p. 159, 1984.

[14] M. Fowler, UML Distilled: A Brief Guide to the Standard Object Modeling Language, 3rd ed. Addison-Wesley, 2003.

[15] Central Motor Vehicles Rules, 1989, and AIS-140 Intelligent Transportation Systems standard, Ministry of Road Transport and Highways, Government of India.

[16] Government of Goa, “Goa Comprehensive Mobility Plan (CMP).”

[17] The Times of India, reports on Goa bus ridership and the Kadamba Transport Corporation fleet.

[18] Down To Earth, transit analysis for Goa.

APPENDIX A: SOURCE CODE LISTINGS & CONFIGURATION FILES

Selected listings are reproduced below. The complete source is in the project repository (`bus-tracker-web`: `backend` and `BusTrackerApp`). Secrets have been removed.

A.1 SecurityConfig.java

```
package com.bustracker;

@Configuration
@EnableWebSecurity
public class SecurityConfig {
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/**").permitAll()
                .anyRequest().authenticated()
            );
        return http.build();
    }
}
```

A.2 WebSocketConfig.java (abridged)

```
@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    @Override
    public void configureMessageBroker(MessageBrokerRegistry registry) {
        registry.enableSimpleBroker("/topic");
        registry.setApplicationDestinationPrefixes("/app");
    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws").setAllowedOriginPatterns("*").withSockJS();
        registry.addEndpoint("/ws").setAllowedOriginPatterns("*");
    }
    // plus interceptor and event listeners that log CONNECT, SUBSCRIBE and
DISCONNECT
}
```

A.3 BusTrackerService.java

```
@Service
public class BusTrackerService {
    private final Set<String> activeBuses = ConcurrentHashMap.newKeySet();

    public void registerActiveBus(String busId) { activeBuses.add(busId); }
    public Set<String> getActiveBuses() { return activeBuses; }

    public double calculateDistanceKm(double lat1, double lon1, double lat2,
double lon2) {
        final int EARTH_RADIUS_KM = 6371;
        double dLat = Math.toRadians(lat2 - lat1);
        double dLon = Math.toRadians(lon2 - lon1);
        double a = Math.sin(dLat / 2) * Math.sin(dLat / 2)
                 + Math.cos(Math.toRadians(lat1)) *
Math.cos(Math.toRadians(lat2))
                 * Math.sin(dLon / 2) * Math.sin(dLon / 2);
        double c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
```

```
        return EARTH_RADIUS_KM * c;
    }

    public long calculateEtaMinutes(double distanceKm) {
        double averageSpeed = 25.0; // km/h
        return Math.round((distanceKm / averageSpeed) * 60);
    }
}
```

A.4 application.properties (secrets masked)

```
server.port=8080
spring.datasource.url=jdbc:h2:file:./data/bustracker;AUTO_SERVER=TRUE
spring.datasource.username=sa
spring.datasource.password=
spring.datasource.driver-class-name=org.h2.Driver
admin.username=admin
admin.password=<set a strong value; use an environment variable>
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.H2Dialect
spring.mail.username=<smtp user>
spring.mail.password=<smtp app password>
```

A.5 Dockerfile

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY . .
RUN mvn clean package -DskipTests

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

A.6 Android build configuration

```
// Android app/build.gradle.kts (key parts)
namespace = "com.project.bus_tracker"; minSdk = 24; targetSdk = 36
implementation("com.google.android.gms:play-services-location:21.3.0")
implementation("com.squareup.okhttp3:okhttp:4.12.0")
implementation("com.squareup.okhttp3:okhttp-sse:4.12.0")
implementation("org.osmdroid:osmdroid-android:6.1.20")
// Jetpack Compose BOM, Material 3, activity-compose, lifecycle
```

APPENDIX B: API DOCUMENTATION & ENDPOINT SPECIFICATIONS

All REST endpoints use JSON. Authentication is a cookie-based HTTP session (`JSESSIONID`). Error responses have the form `{ "status": "ERROR", "message": "..." }`. The live version of this documentation is served by the application at `/docs`.

Table B.1: REST and WebSocket interface summary

Method	Path	Purpose	Main responses
POST	/api/auth/signup	Create passenger or driver account (driver phone must be whitelisted)	201, 400, 403, 409
POST	/api/auth/request-otp	Request OTP for a whitelisted driver phone	200, 400, 403
POST	/api/auth/login	Passenger login (username, password) or driver login (phone, category, OTP); aliases `/driver/login`, `/verify-otp`	200, 400, 401, 403
POST	/api/auth/logout	Invalidate session	200
WS	/ws (SEND /app/update-location)	Driver publishes a BusLocationUpdate	Broadcast to SSE and STOMP topics
WS topics	/topic/bus/{busId}, /topic/bus-locations, /topic/active-buses, /topic/bus-stopped, /topic/schedule-added	Subscribed by clients for live events	STOMP messages
POST	/api/buses/stop-sharing	Driver stops sharing; body `{ "busId": "..." }`	200, 400
GET	/api/buses/active	IDs of active buses	200
GET	/api/buses/locations	Current live locations	200
GET	/api/buses/locations/stream?clientId=	Open SSE stream (events: location-update, bus-stopped, heartbeat)	text/event-stream
POST	/api/buses/track	Track a bus for a client (clientId, busId)	200
POST	/api/buses/untrack	Stop tracking a bus	200
GET	/api/buses/all-scheduled-locations	Scheduled start positions merged with live positions	200
POST	/api/admin/login	Admin login	200, 401
POST	/api/admin/logout	Admin logout	200
GET	/api/admin/status	Whether the session is an admin session	200
GET / POST	/api/admin/whitelist	List / add whitelisted phone number	200, 201, 401
DELETE	/api/admin/whitelist/{phoneNumber}	Remove a phone number	200, 401
GET	/api/admin/schedules	List schedules	200

Method	Path	Purpose	Main responses
POST	/api/admin/schedules	Create schedule, generate driver password, e-mail credentials	200, 400, 500
PUT	/api/admin/schedules/{scheduleId}	Update schedule	200
DELETE	/api/admin/schedules/{scheduleId}	Delete schedule	200
GET	/admin, /docs	Admin web page; API documentation page	200

Example driver login request:

```
POST /api/auth/login
{ "phoneNumber": "+919876543210", "busCategory": "KTCL", "otp": "<code>" }

200 OK
{ "status": "SUCCESS", "role": "DRIVER", "busId": "...", "driverName": "...",
  "phoneNumber": "+919876543210", "busCategory": "KTCL" }
```

Example location update (STOMP SEND to `/app/update-location`):

```
{ "busId": "GA-03-X-1001", "busNumber": "1001", "latitude": 15.5937, "longitude":
73.8142,
  "startPoint": "Panaji", "destination": "Sanquelim",
  "routeTaken": "Panaji-Mapusa-Bicholim-Sanquelim", "busCategory": "KTCL" }
```

APPENDIX C: RAW SURVEY DATA & USER TESTING CHECKLISTS

C.1 Commuter survey questionnaire

1. How often do you use public buses in Goa? (daily / weekly / occasionally)
2. Which type of bus do you mostly use? (KTCL / private / both)
3. How long do you usually wait for a bus? (under 10 min / 10 to 20 / 20 to 45 / more)
4. Do you currently use any app to find buses? Which one?
5. Would you use an app that shows live bus locations? (yes / maybe / no)
6. Which feature matters most? (live map / ETA / notifications / route search)
7. Would you pay for an ad-free version? (yes / no)

C.2 User acceptance testing checklist

Table C.1: UAT checklist

Task	Completed (Y/N)	Time taken	Comments
Passenger: sign up and log in	Y	~45 s	Account creation returned HTTP 201; successful initial authentication.
Passenger: find and filter buses	Y	~15 s	Route filter accurately dynamically isolates KTCL and Private buses.
Passenger: track and untrack a bus	Y	~20 s	SSE stream establishes immediately; marker un-tracks without residual events.
Driver: log in with whitelisted number	Y	~30 s	Phone whitelist check and OTP validation completed on first attempt.
Driver: start and stop sharing	Y	~25 s	Location permission prompt handled; foreground notification active during sharing.
Admin: whitelist a number	Y	~10 s	Number added via web dashboard and immediately made available for driver auth.
Admin: create, edit, delete a schedule	Y	~40 s	Schedule entry created with automated driver notification email; CRUD actions verified.

C.3 Satisfaction questionnaire (1 = strongly disagree, 5 = strongly agree)

- The app was easy to learn and use.
- The bus positions on the map were accurate and up to date.
- The category filter and side panel helped me find a bus.
- I would use this app in my daily commute.
- I would recommend this app to others.